

Software Technology 2 Group Assignment (7170 UG)

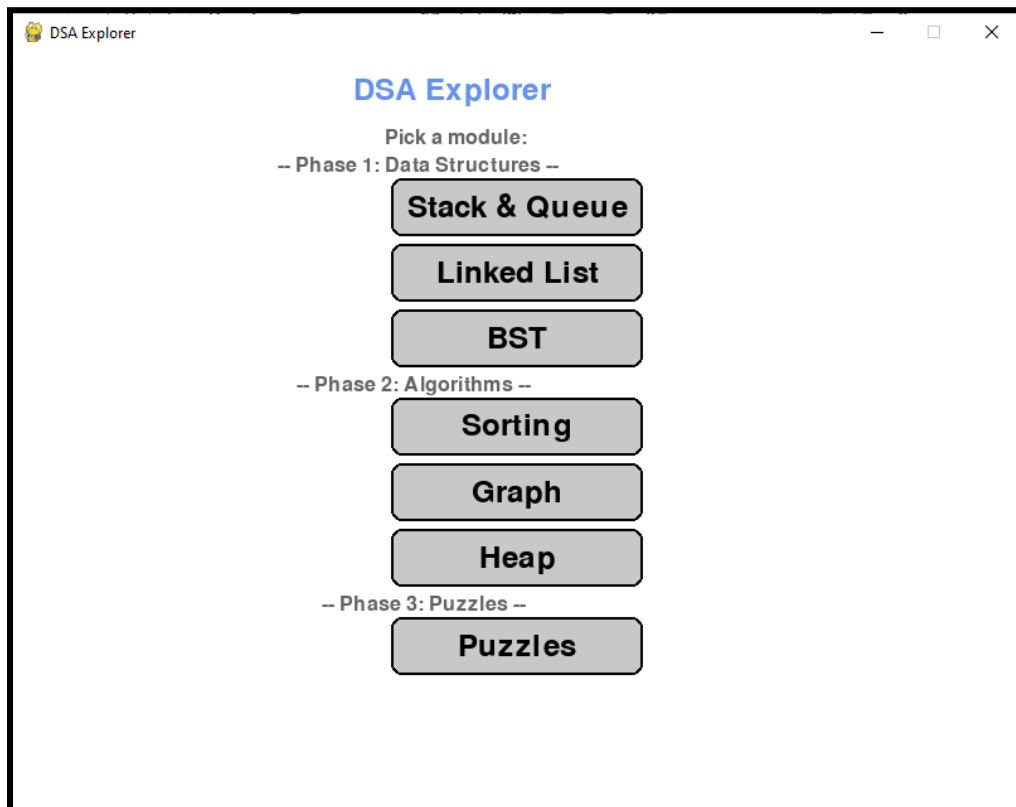
Haider Balouch - u3293869

Zubair Akhtar - u3306240

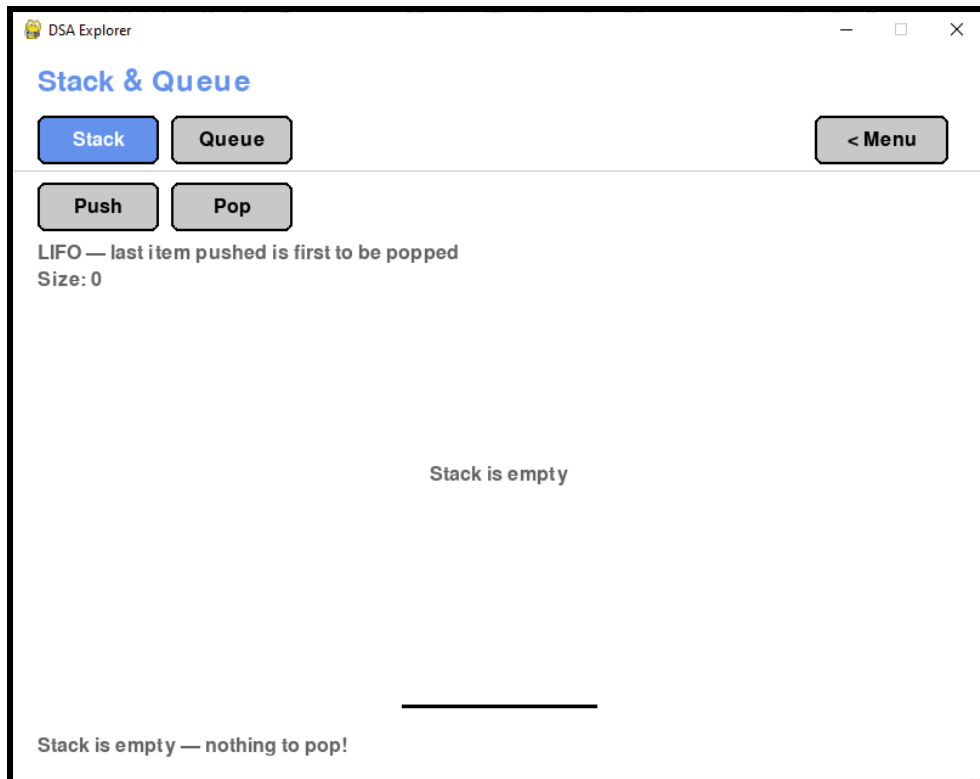
Phase 1:

Visual Stack and Queue:

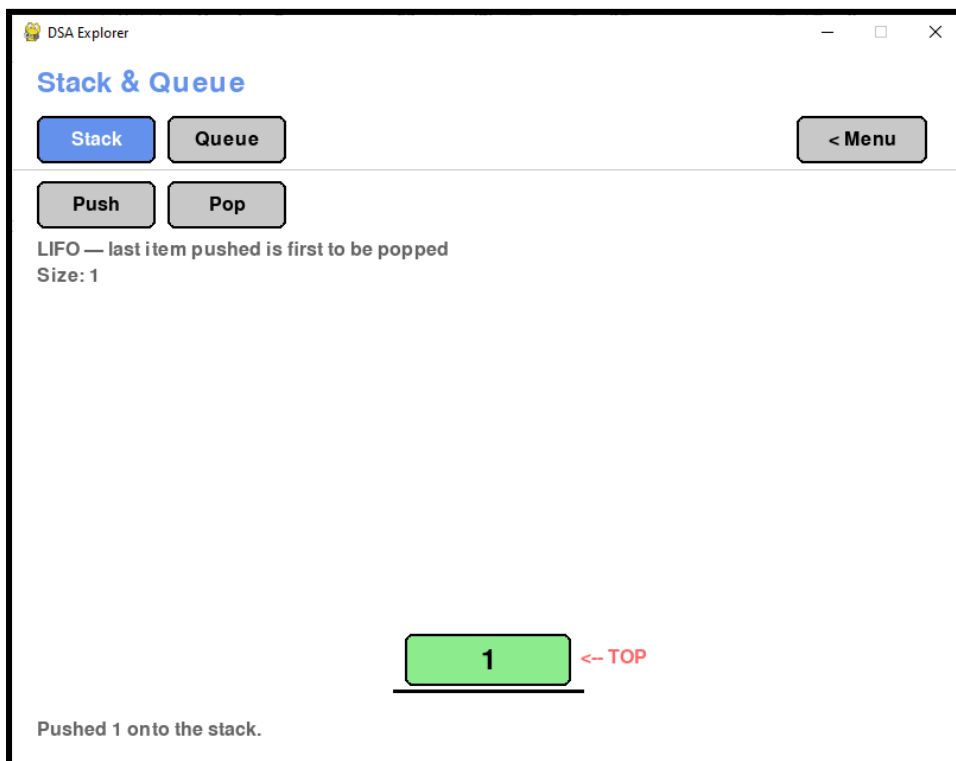
Stack:



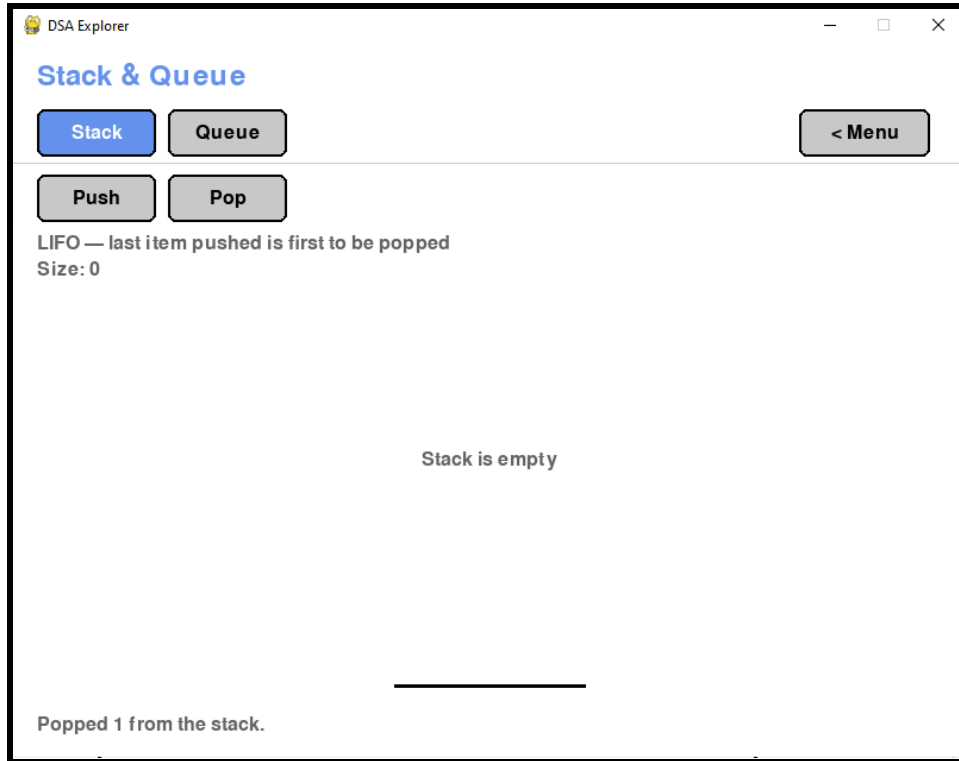
(Figure 1 - Main Menu of application)



(Figure 2a - Opening Stack & Queue: Stack Menu)



(Figure 3 - Push)

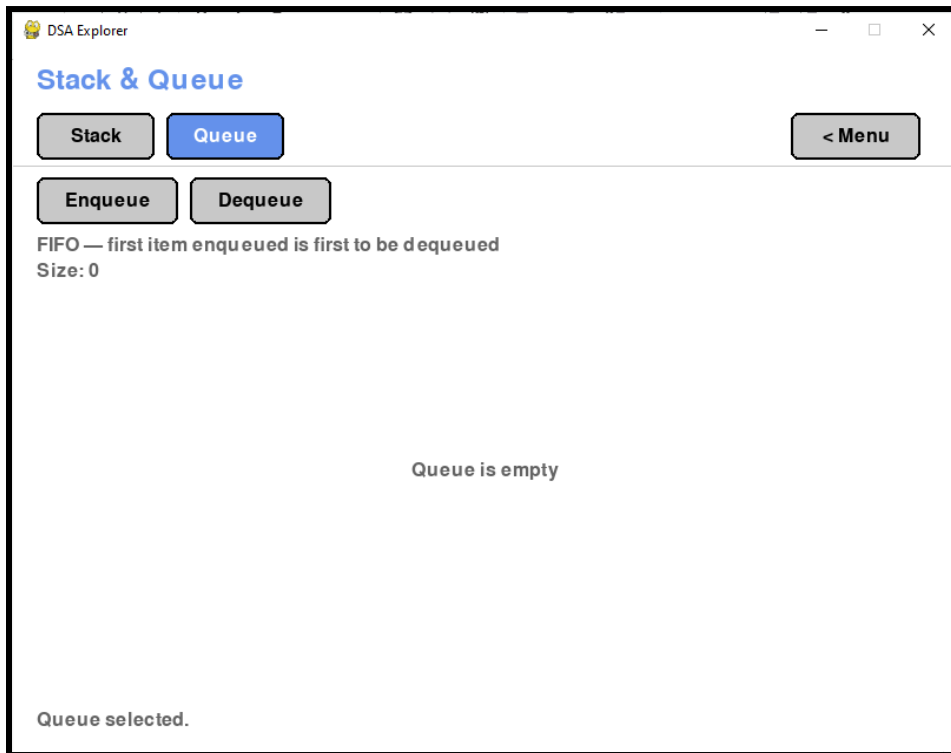


(Figure 4 - Pop)

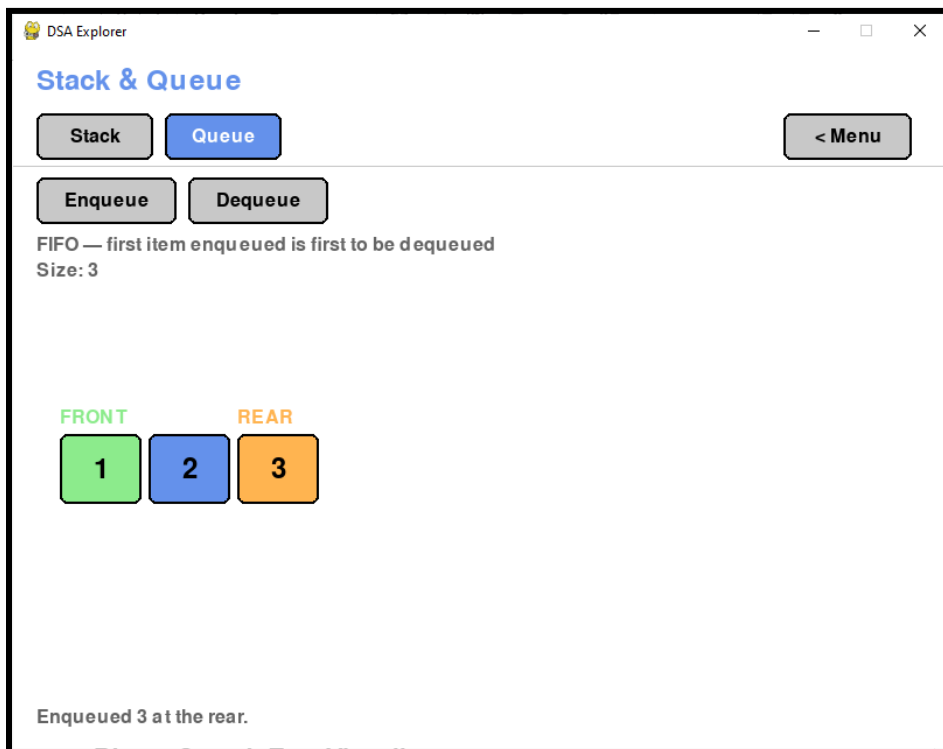
Description:

The Stack module demonstrates a Last-In-First-Out (LIFO) data structure, where the most recently pushed element is always the first to be removed. Figures 2a-4 show the stack starting empty, an element being pushed onto the top, and then that element being popped off, leaving the stack empty again.

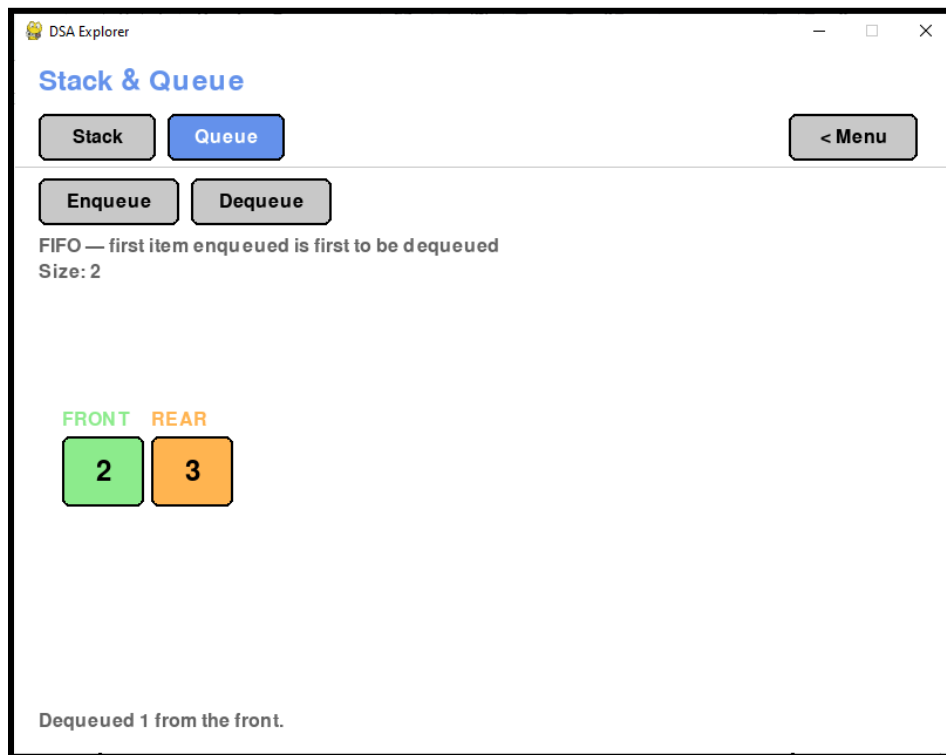
Queue:



(Figure 2b - Opening Stack & Queue: Queue Menu)



(Figure 5 - Enqueue)

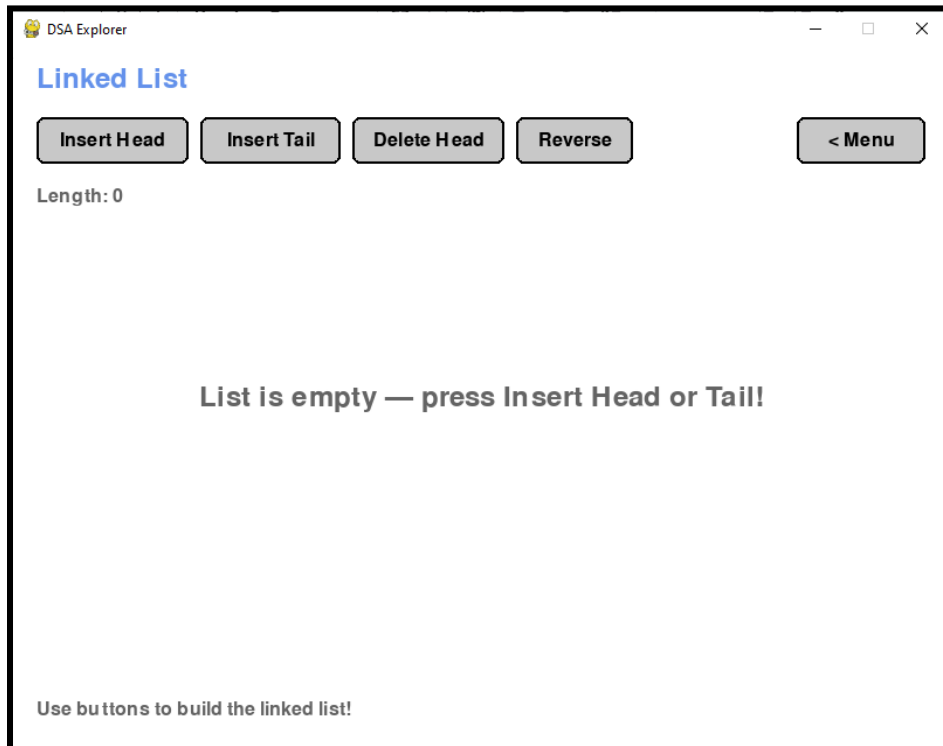


(Figure 6 - Dequeue)

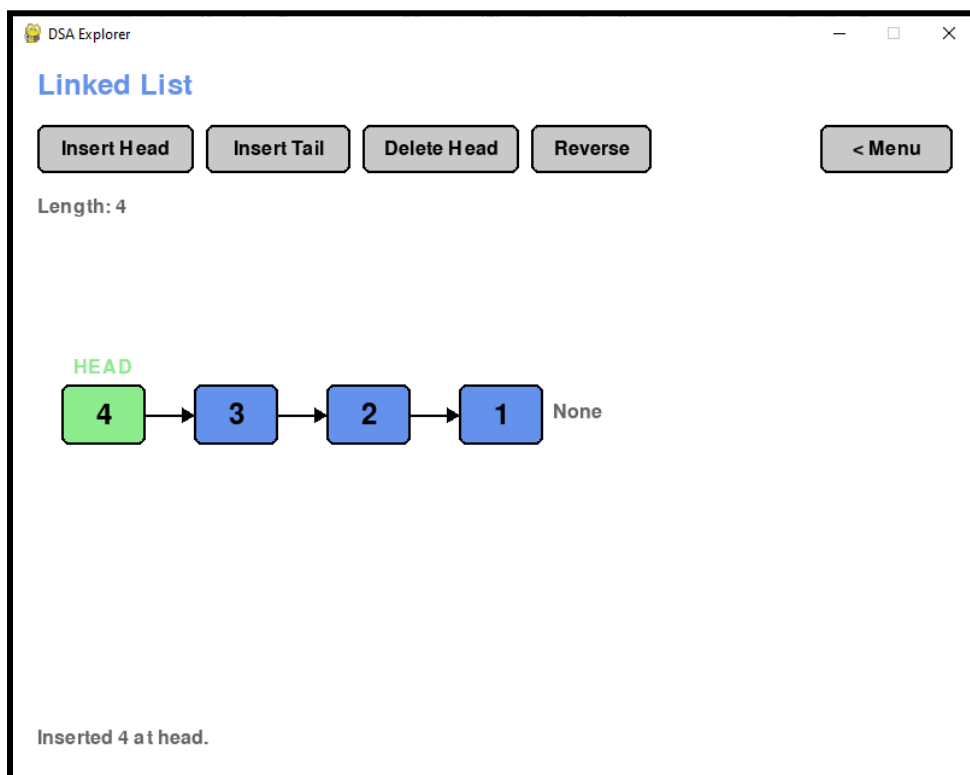
Description:

The Queue model demonstrates a First-In-First-Out (FIFO) data structure, where elements are added at the rear and removed from the front. Figures 2b-6 show the queue being initialized, three elements enqueued one at a time, and then the front element dequeued, reducing the queue size from 3 to 2.

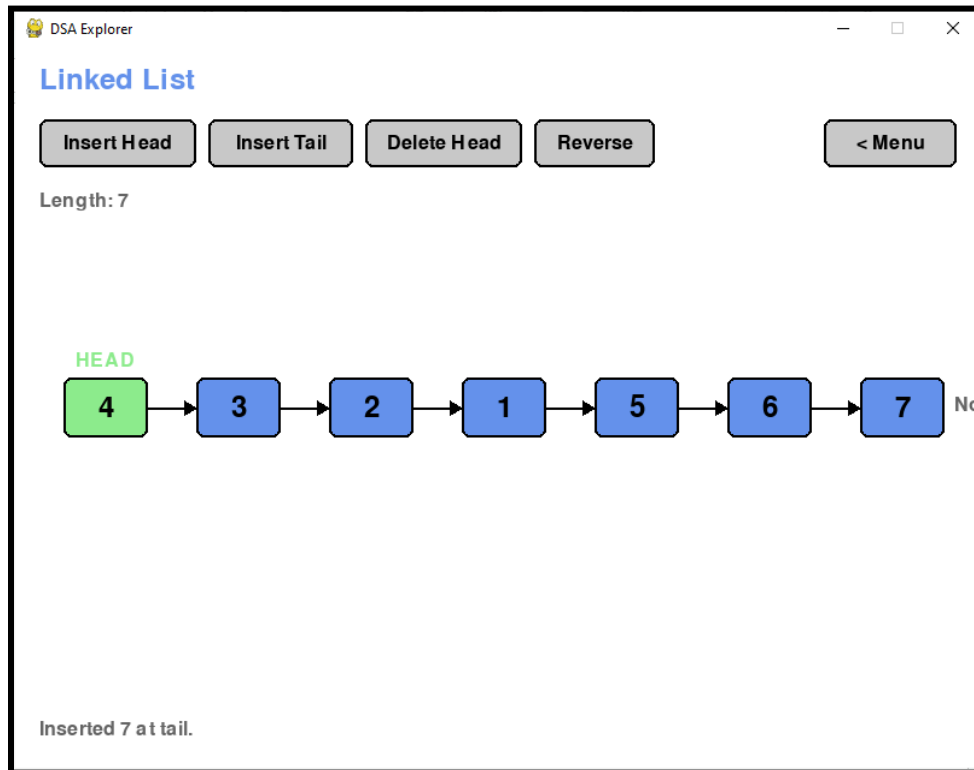
Linked List Editor:



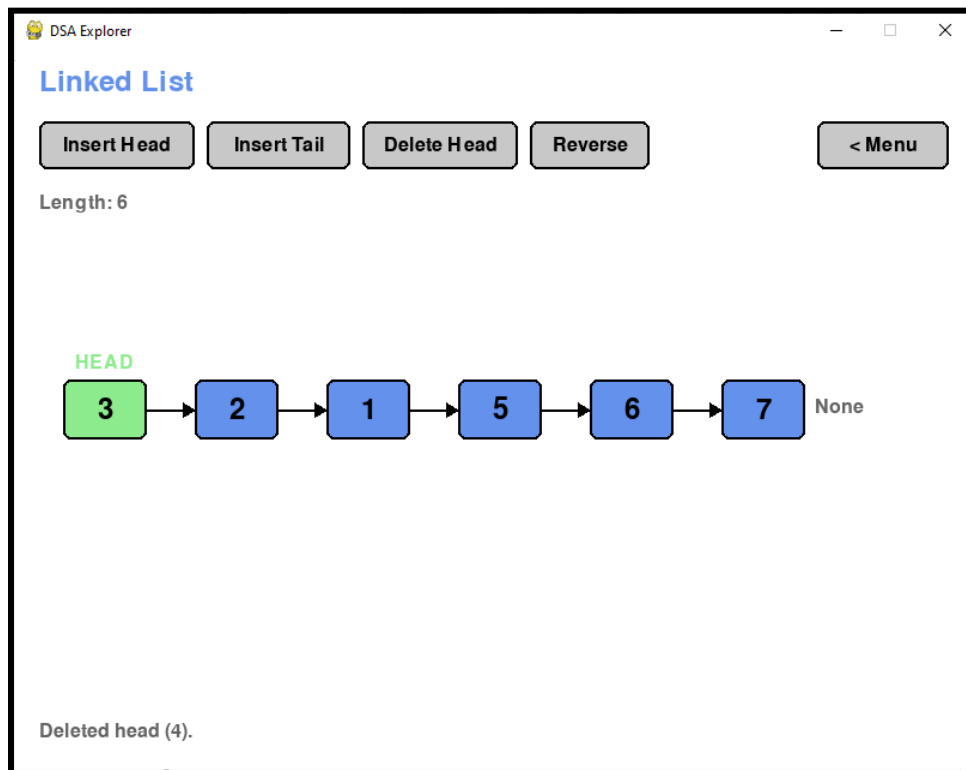
(Figure 7 - Linked List menu)



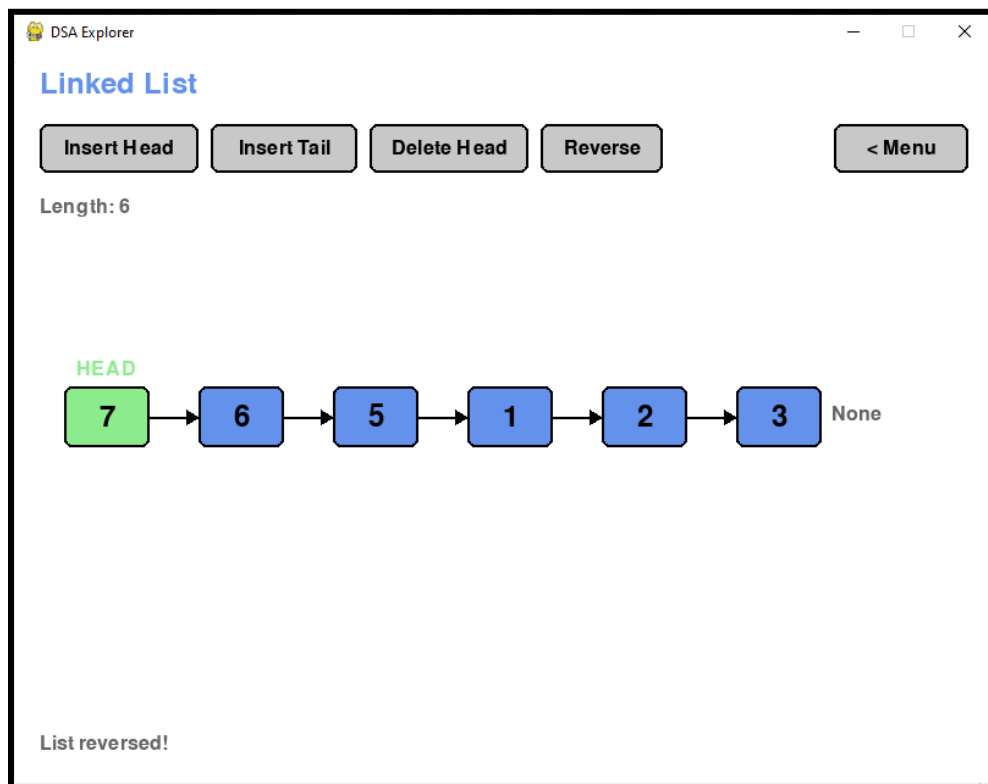
(Figure 8 - Insert Head)



(Figure 9 - Insert Tail)



(Figure 10 - Delete Head)



(Figure 11 - Reverse)

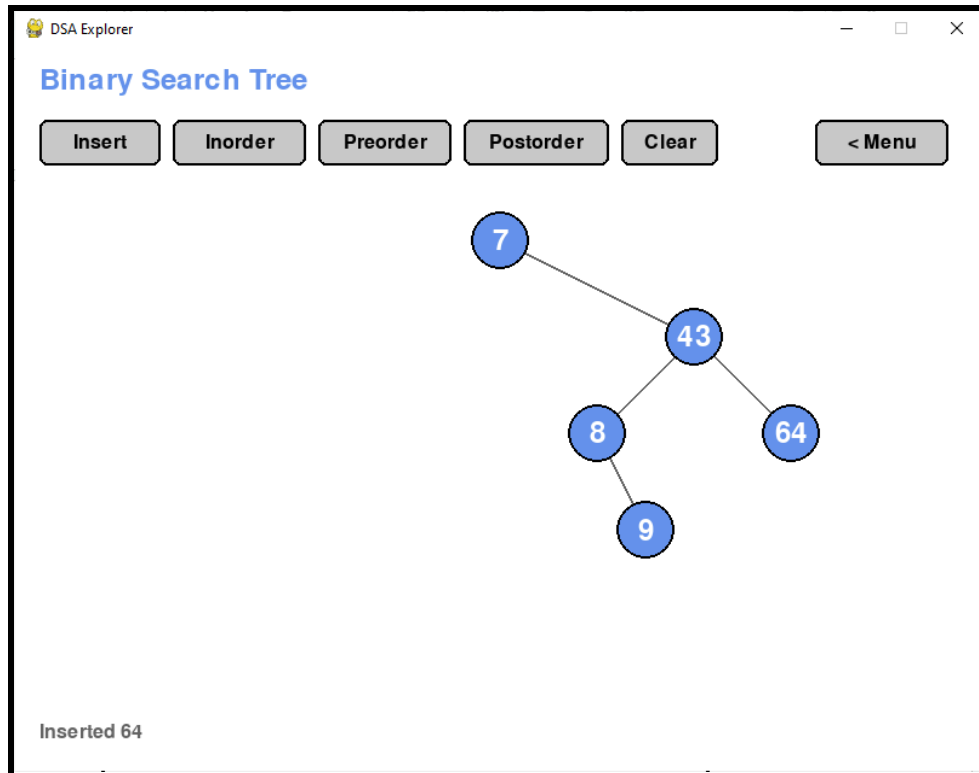
Description:

The Linked List module demonstrates a singly linked list where nodes are connected sequentially via pointers, supporting insertion at the head or the tail, deletion of the head node, and full list reversal. Figures 7-11 show the list being built by inserting values at both ends, the head node being deleted and the next node becoming the new head, and finally the entire list being reversed so that the order of nodes is flipped and the HEAD pointer moves to what was previously the tail.

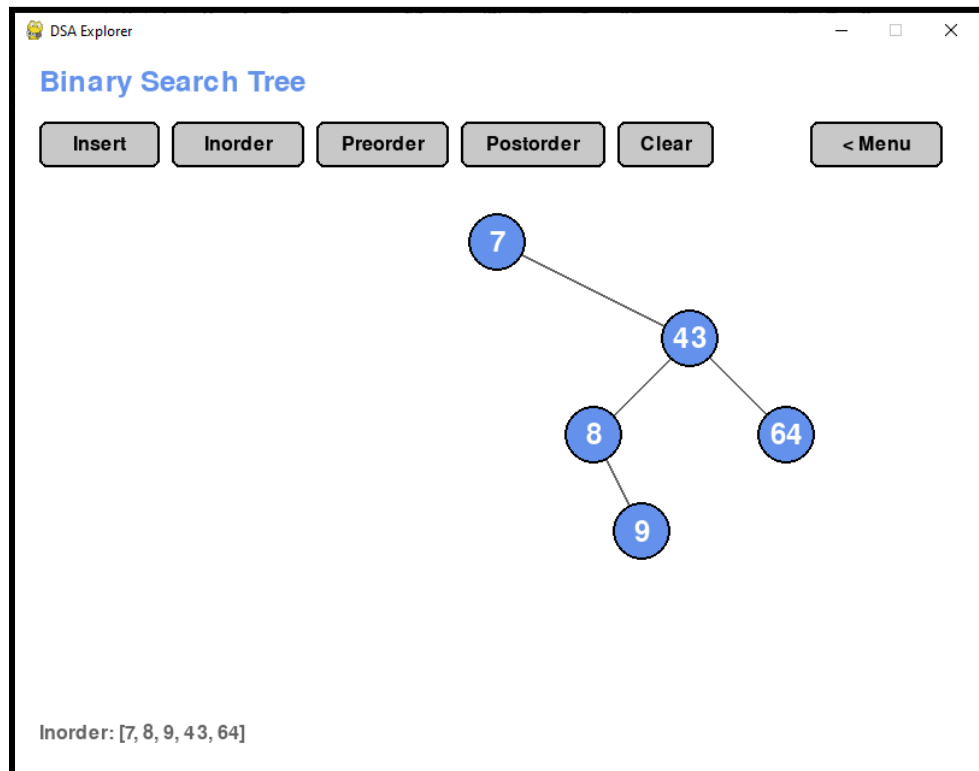
Binary Search Tree Visualiser:



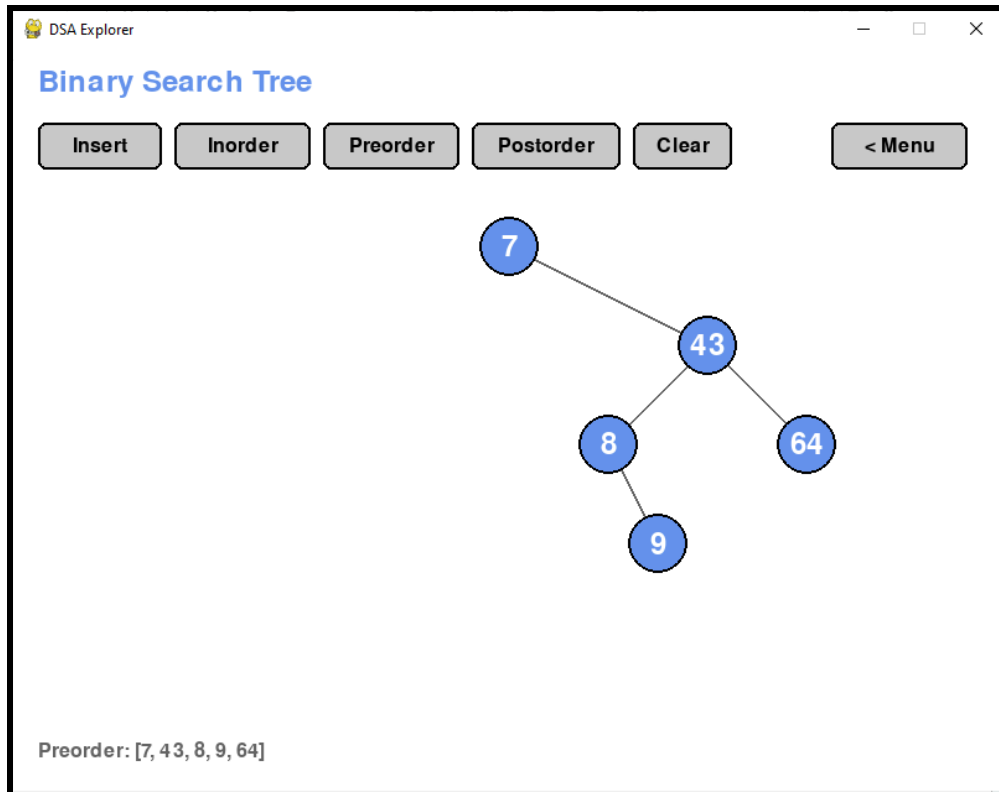
(Figure 12 - BST main menu)



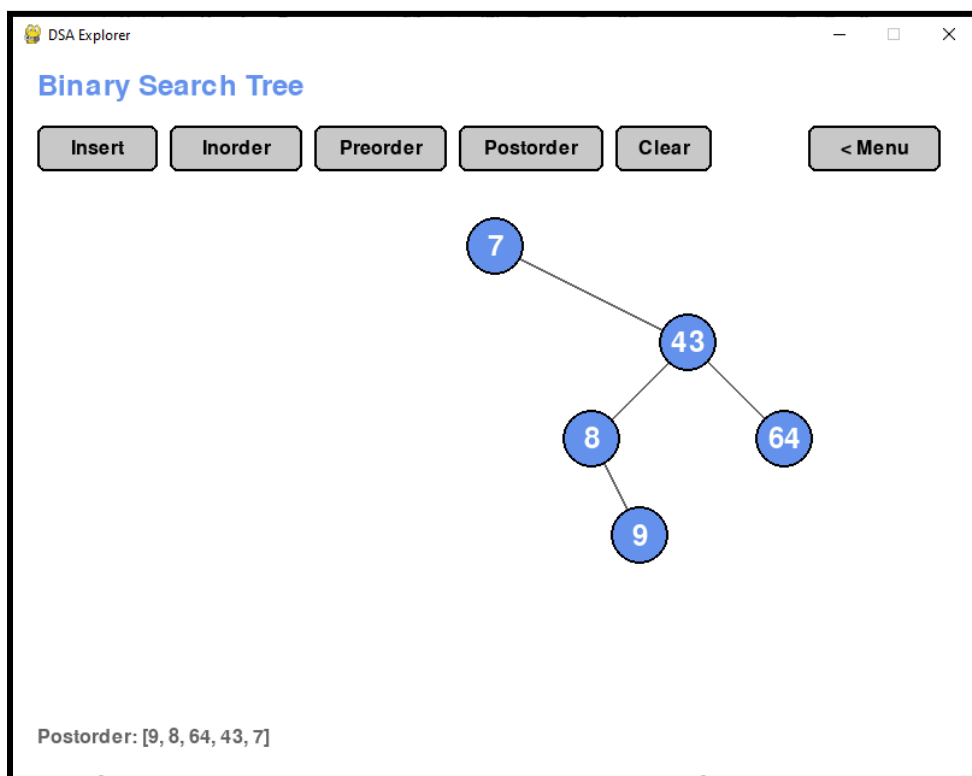
(Figure 13 - Insert)



(Figure 14 - Inorder)



(Figure 15 - Preorder)



(Figure 16 - Postorder)



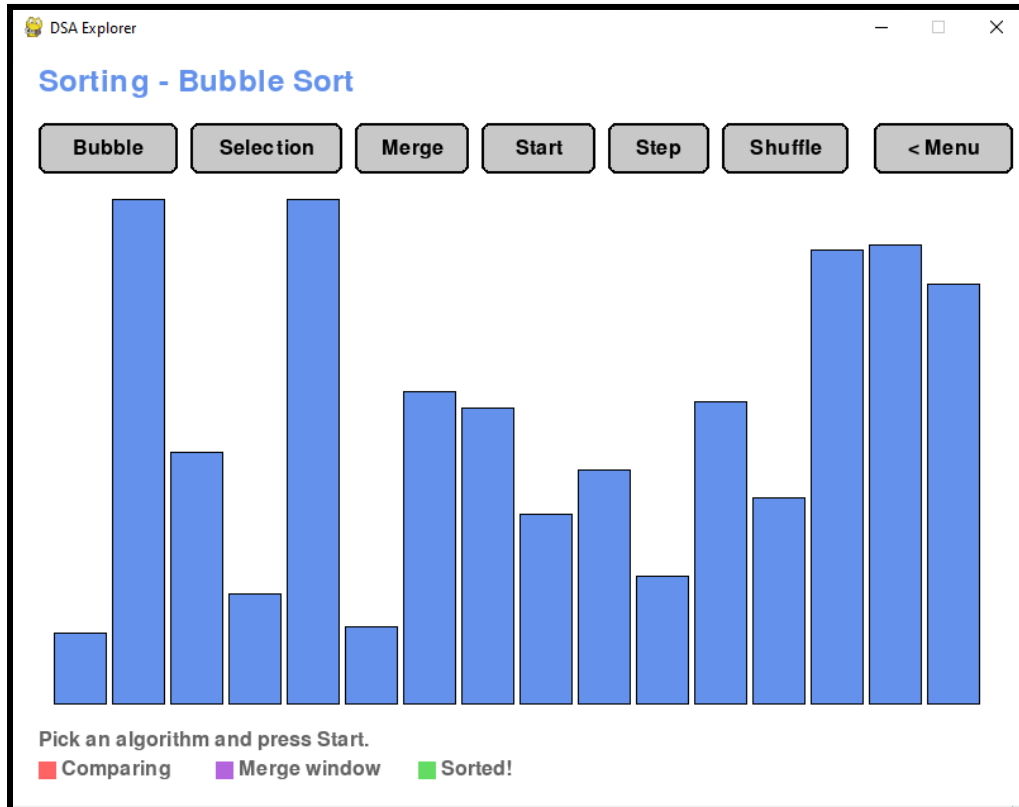
(Figure 17 - Clear)

Description:

The Binary Search Tree module demonstrates how values are inserted and organised so that each node's left child is smaller and its right child is larger, enabling efficient traversal in multiple orders. Figures 12-17 show the tree being built by inserting values, then traversed in inorder (sorted ascending), preorder (root first), and postorder (root last) sequences, before the tree is fully cleared back to its empty state.

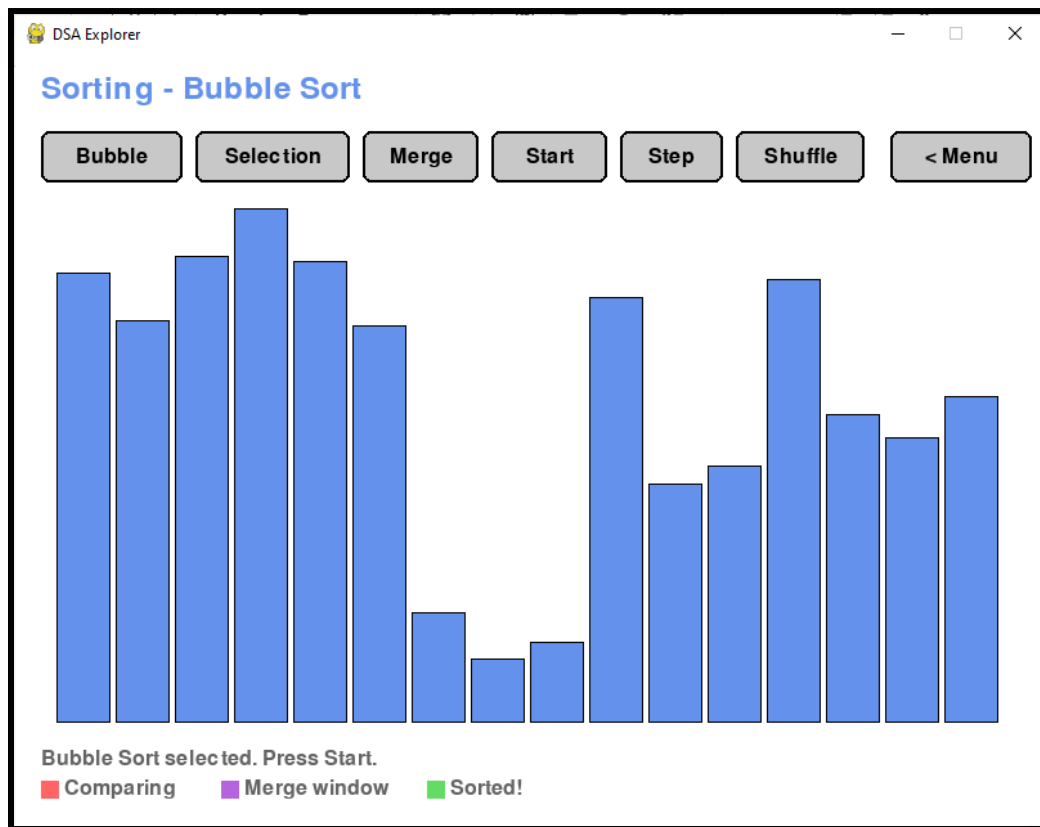
Phase 2:

Sorting Algorithms visualisations:

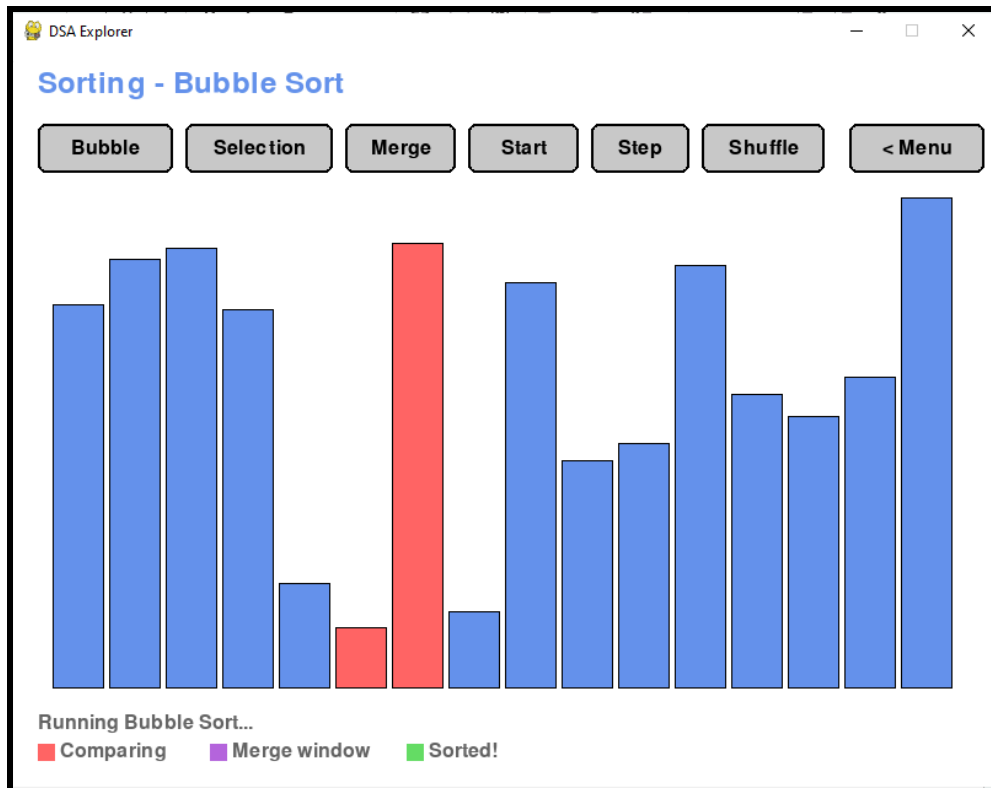


(Figure 18a - Sorting main menu)

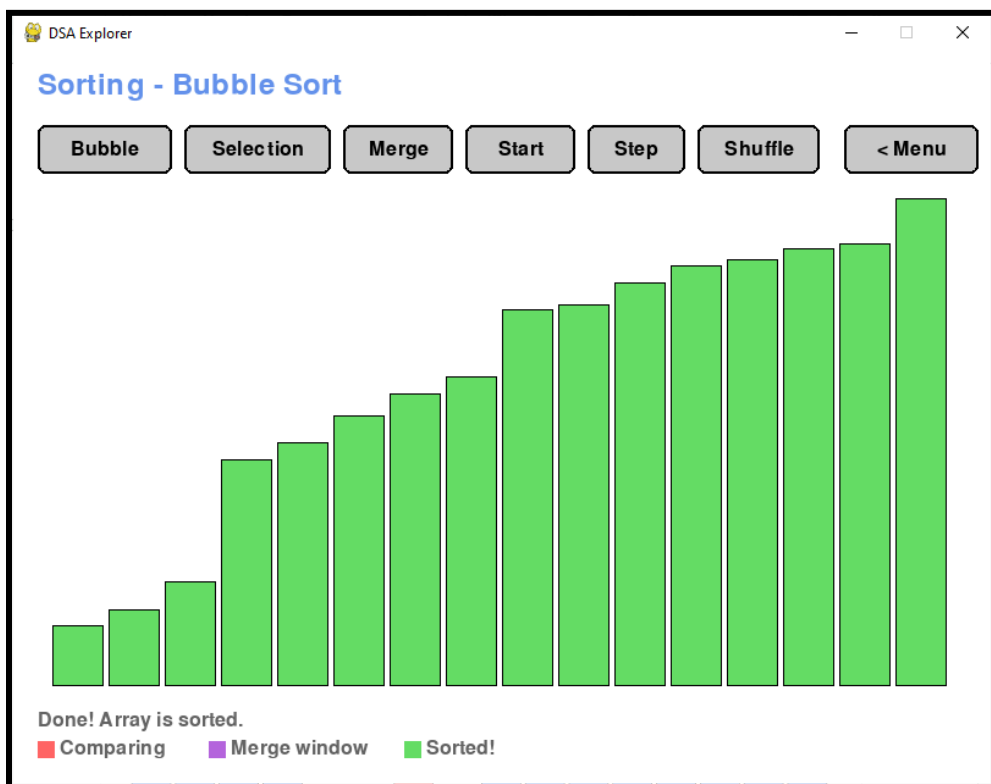
Bubble Sort:



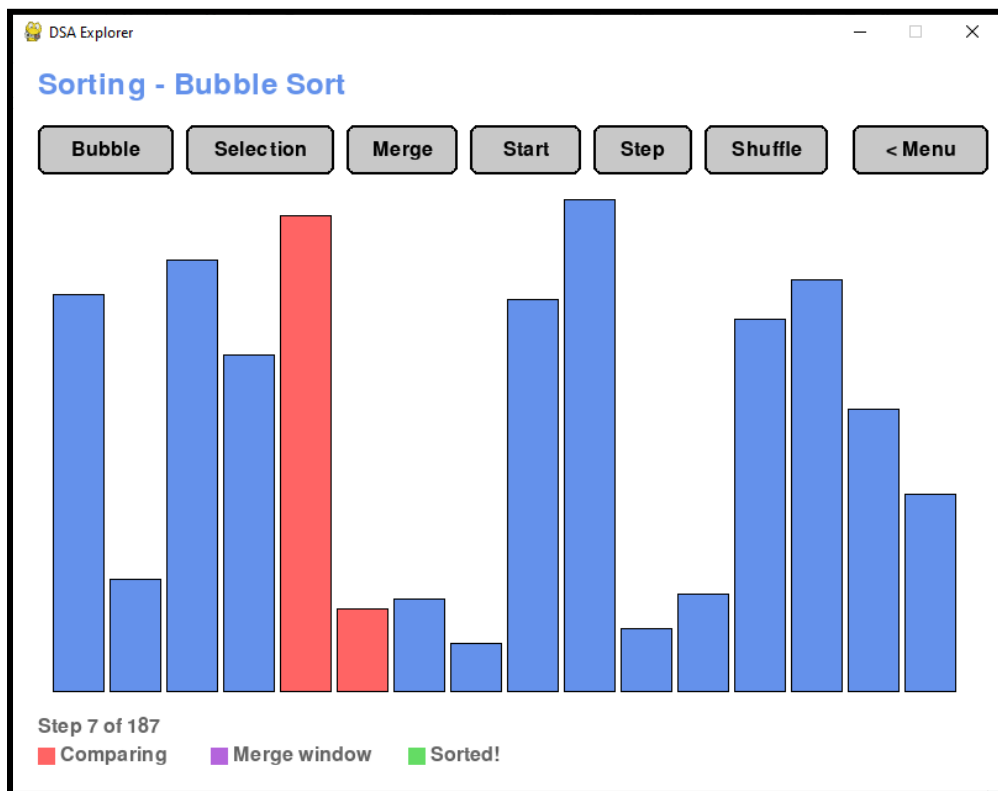
(Figure 18b - Bubble Sort selected)



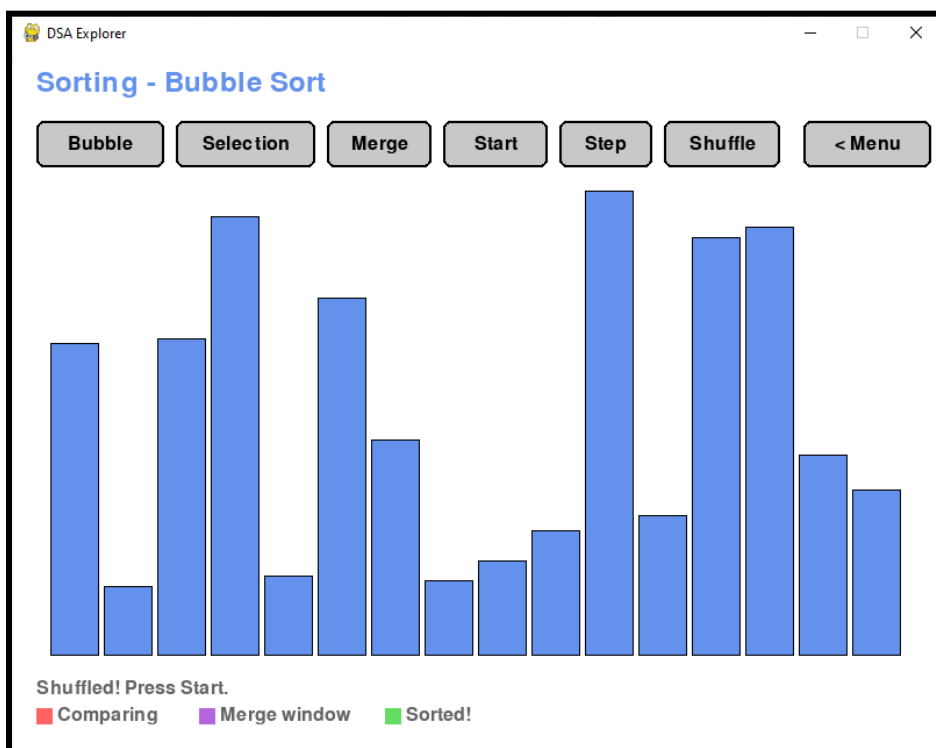
(Figure 19 - Running Bubble Sort)



(Figure 20 - Bubble Sort completed)



(Figure 21 - Step: Bubble Sort)



(Figure 22 - Shuffle)

Description:

The Bubble Sort module visualises the algorithm repeatedly comparing adjacent elements and swapping them if out of order, causing larger values to “bubble” toward the end of the array. Figures 18b-22 show the sort being run to completion with all bars turning green, the Step function advancing one comparison at a time with the active pair highlighted in red, and the Shuffle function, generating a new randomised array ready for another sort (the Shuffle function will work the same across all different sorting algorithms, therefore separate figures for it have not been added in other sections).

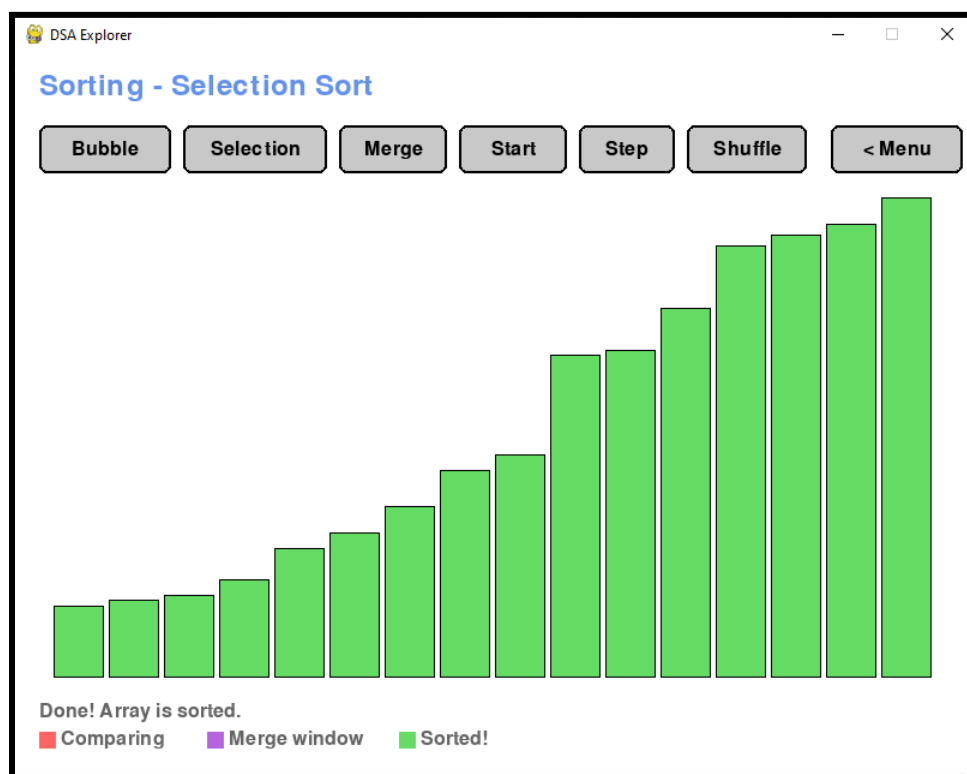
Selection Sort:



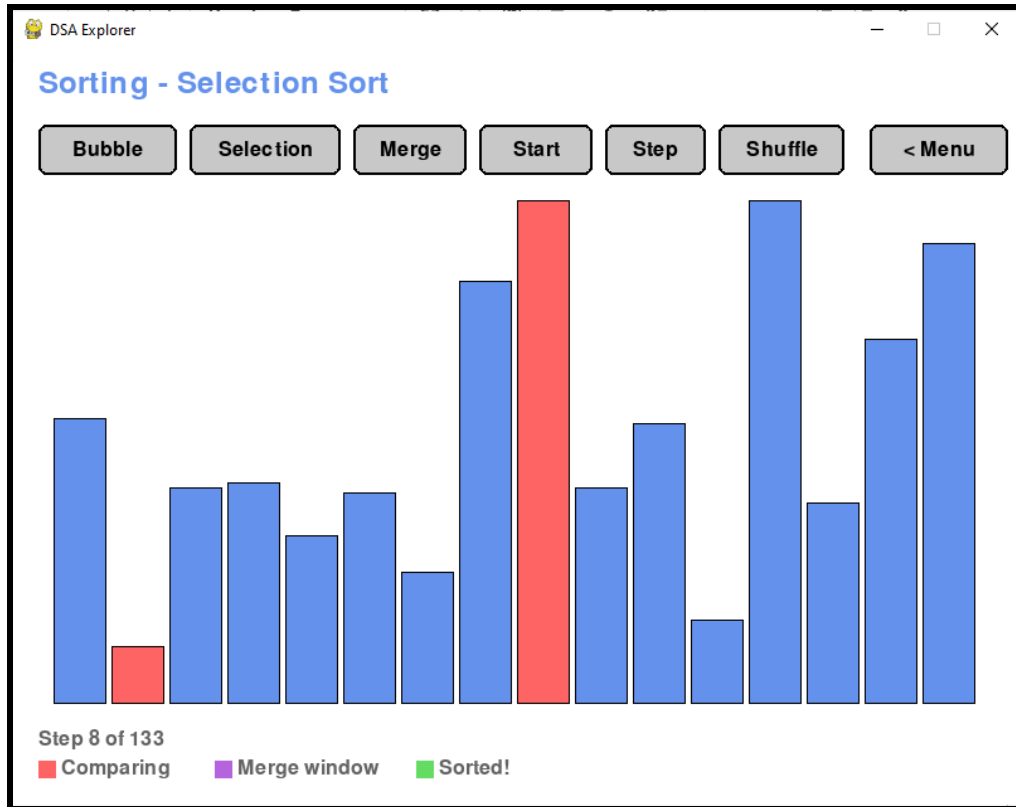
(Figure 18c - Selection Sort selected)



(Figure 23 - Selection Sort running)



(Figure 24 - Selection Sort completed)

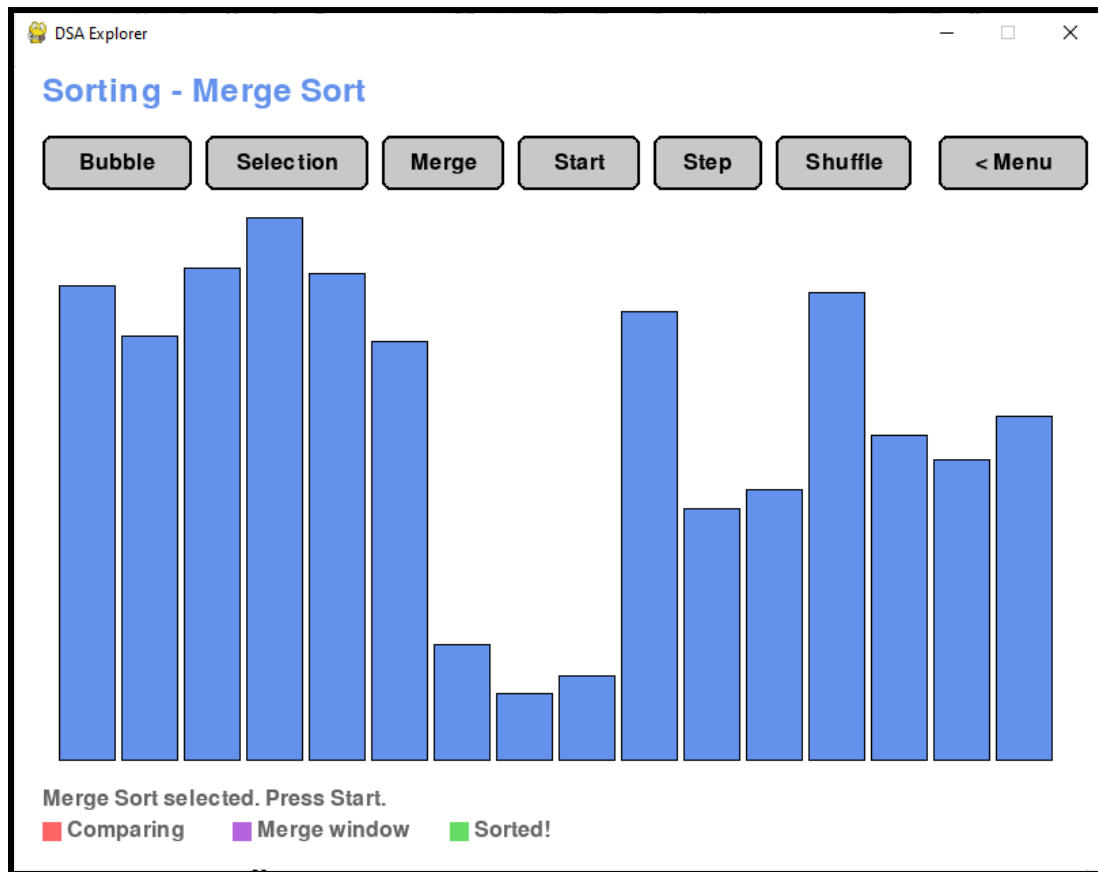


(Figure 25 - Step: Selection Sort)

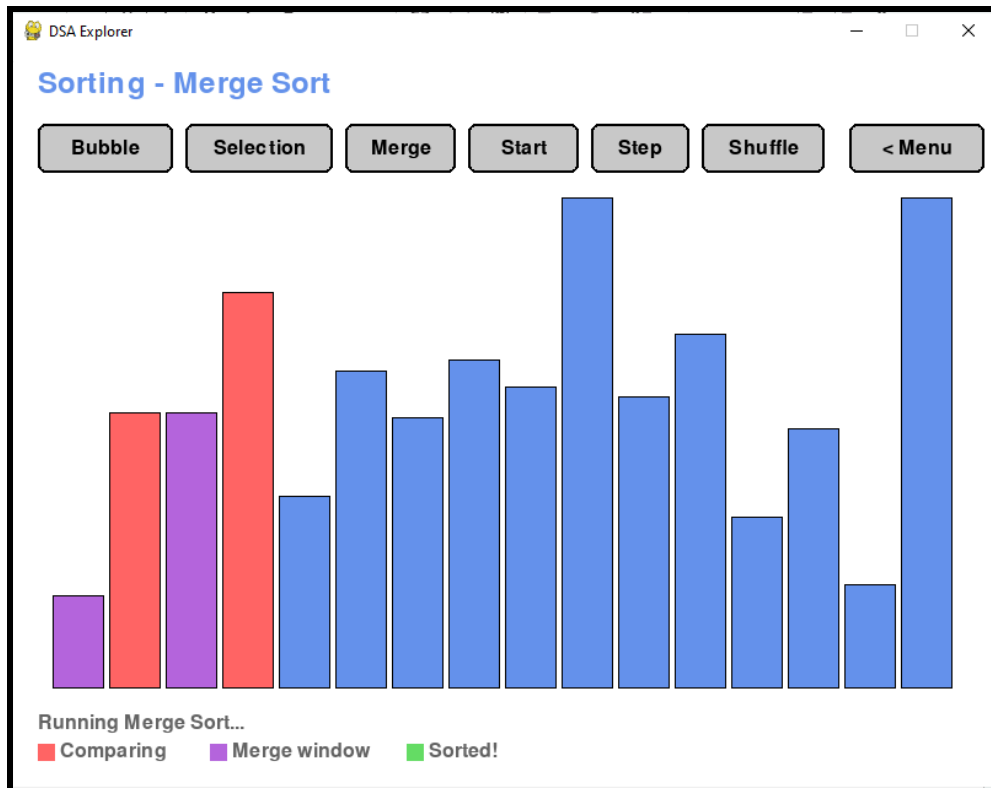
Description:

The Selection Sort module visualises the algorithm scanning the unsorted portion of the array to find the minimum element and placing it into its correct position, one pass at a time. Figures 18c-25 show the sort running with the current minimum highlighted in red, completing when all bars turn green, and the Step function allowing manual progression through each individual comparison.

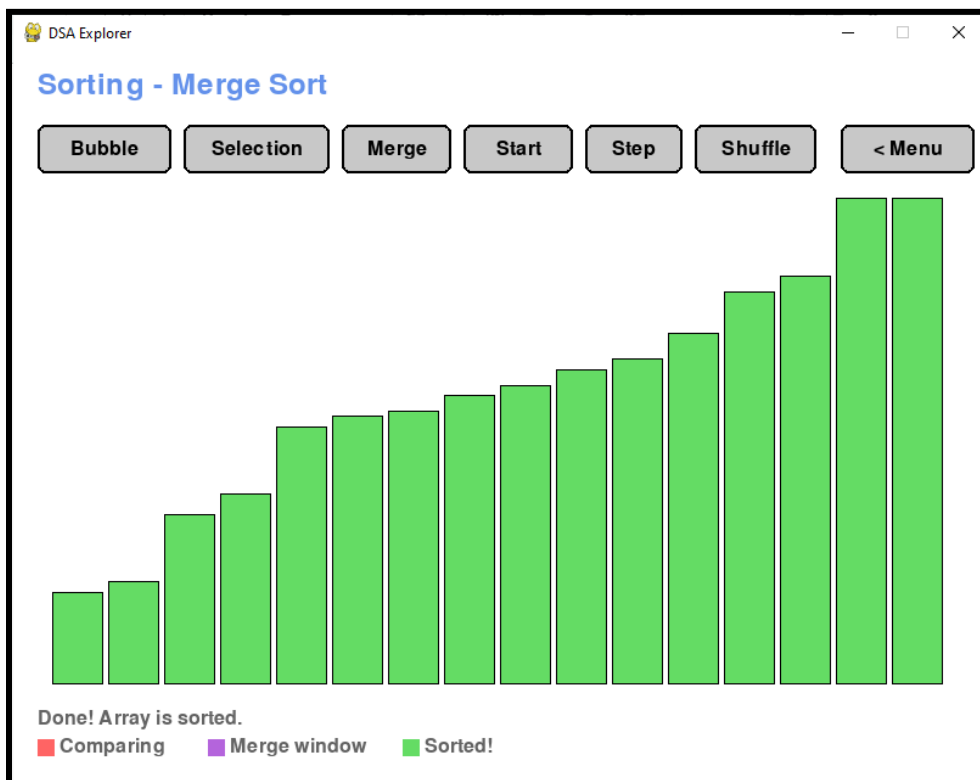
Merge Sort:



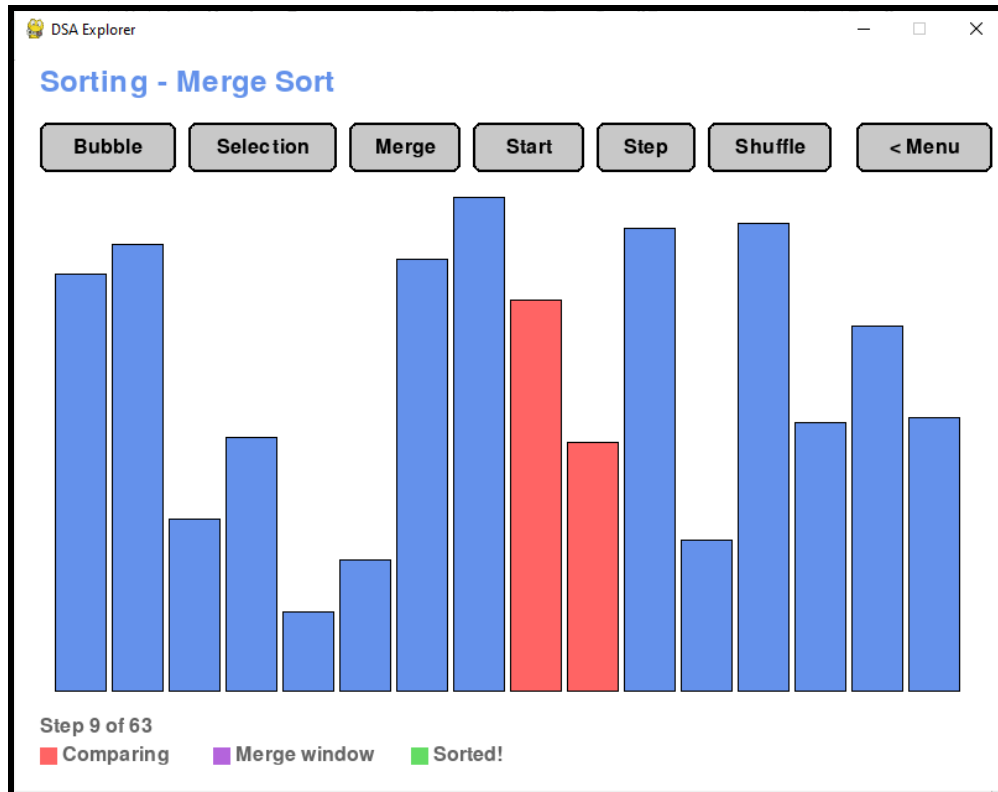
(Figure 18d - Merge Sort selected)



(Figure 26 - Merge Sort running)



(Figure 27 - Merge Sort completed)

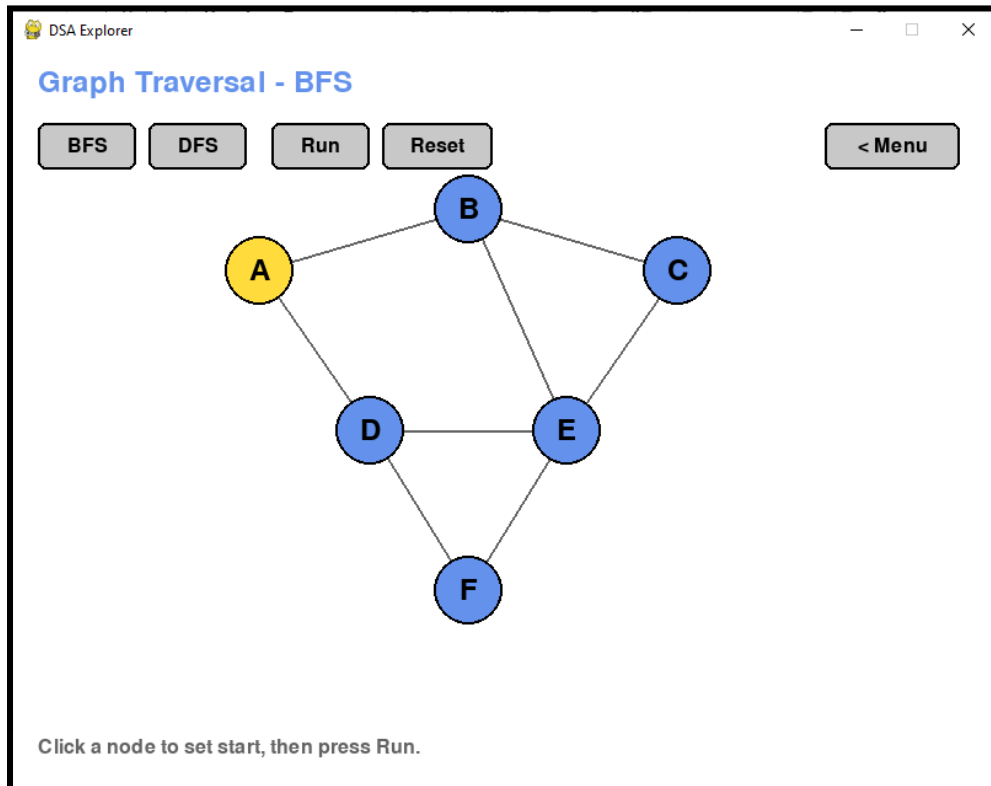


(Figure 28 - Step: Merge Sort)

Description:

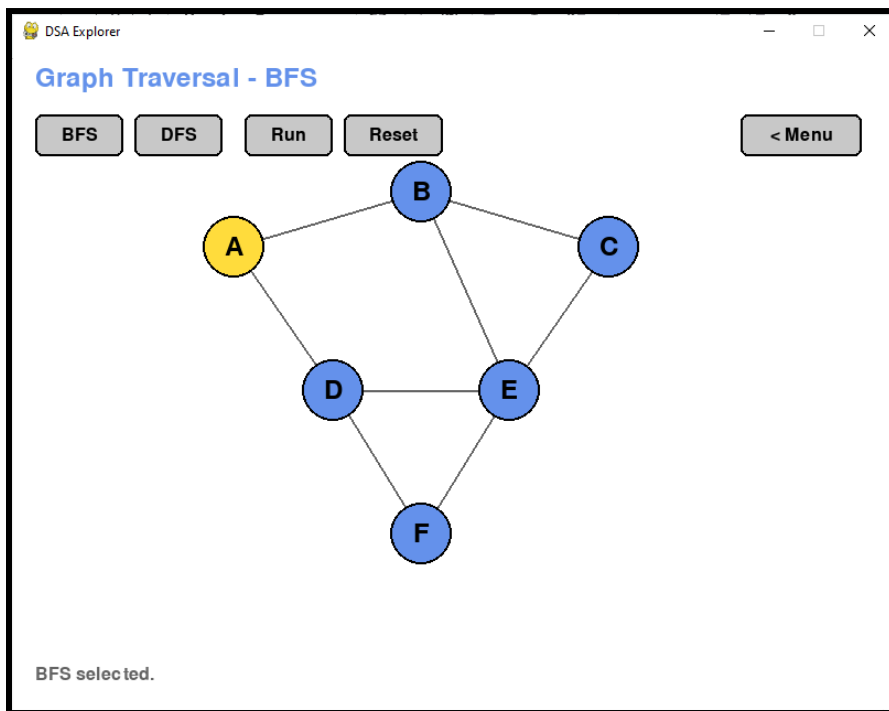
The Merge Sort module visualises the divide-and-conquer algorithm that recursively splits the array into halves, sorts each half, and merges them back together in order. Figures 18d-28 show the sort in progress with the active merge window highlighted in purple and comparing elements in red, completing when all bars are sorted and it turns green, and the Step function displaying each individual merge comparison.

Graph:

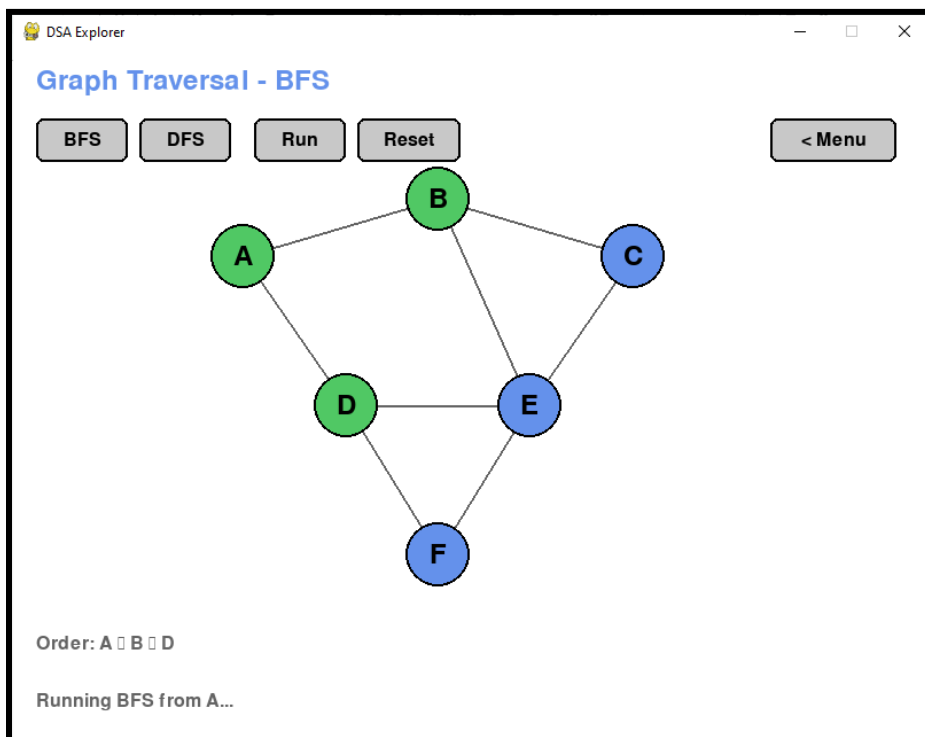


(Figure 29 - Graph main menu)

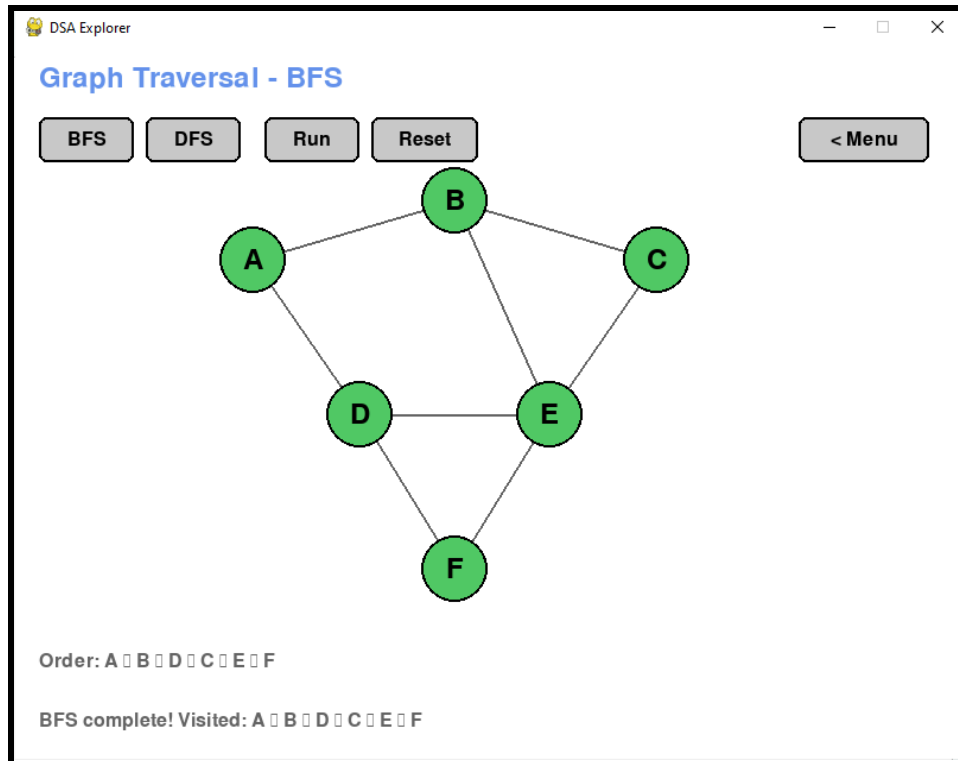
BFS:



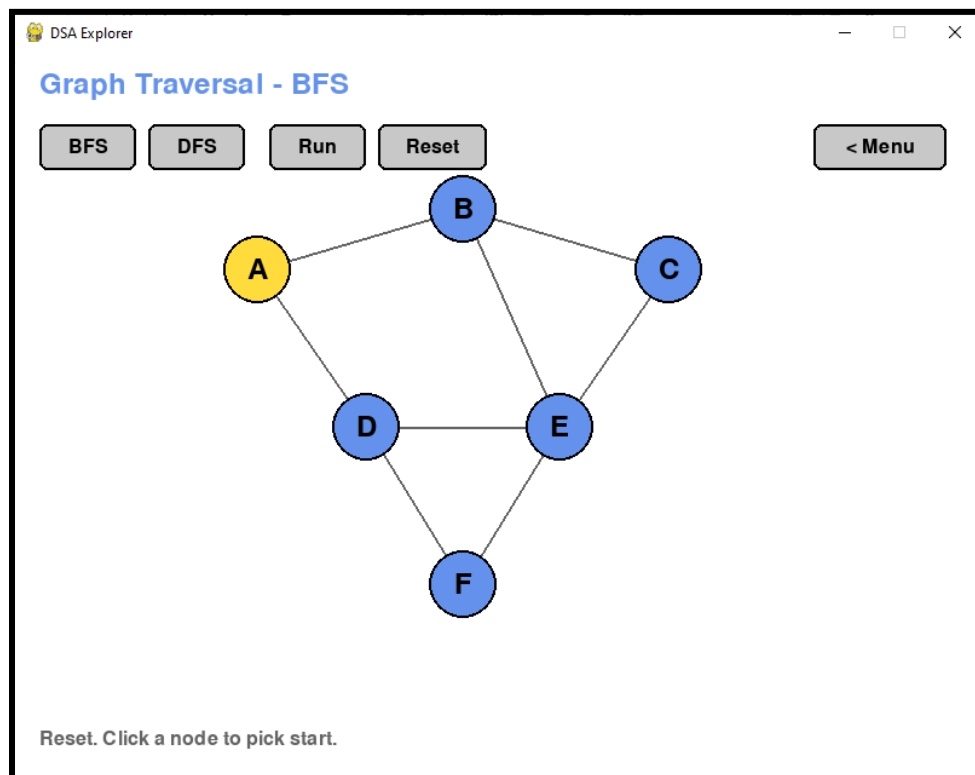
(Figure 30a - BFS selected)



(Figure 31 - BFS running)



(Figure 32 - BFS complete)

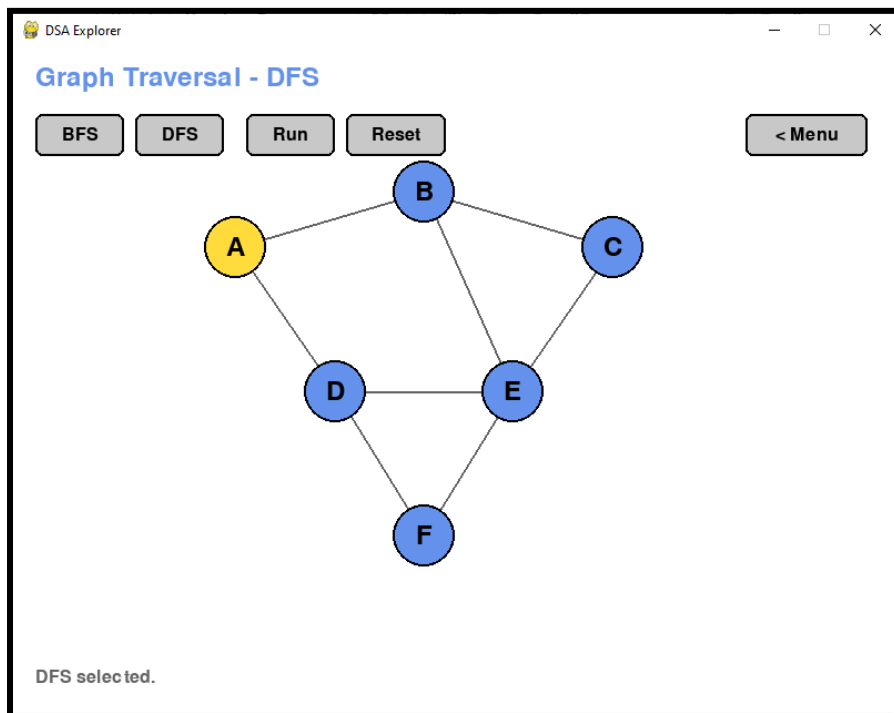


(Figure 33 - BFS reset (same as Figure 36))

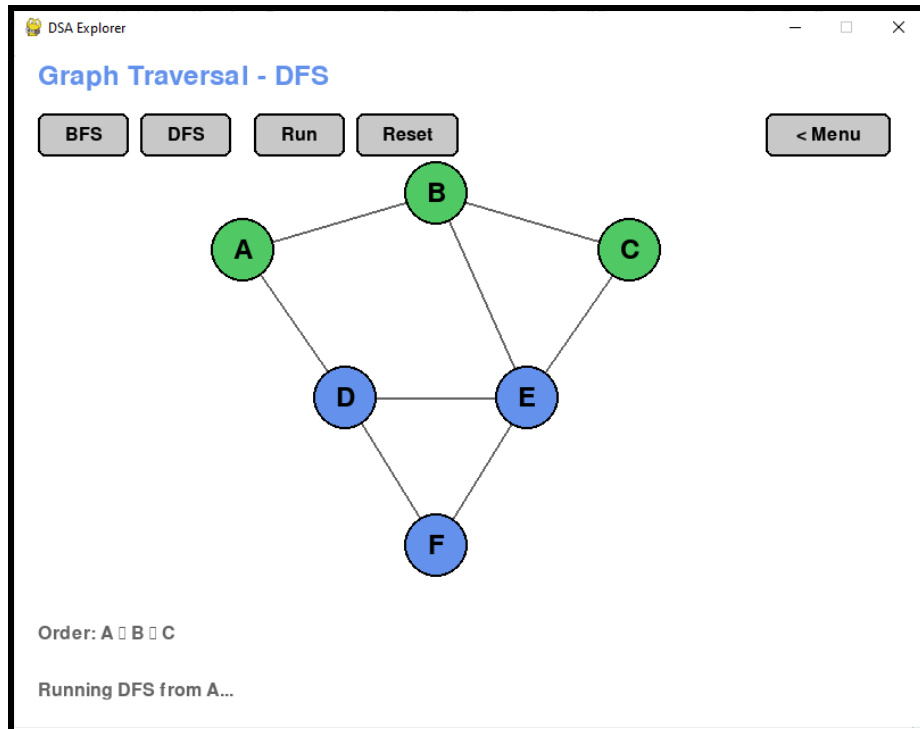
Description:

The BFS module demonstrates Breadth-First Search, which traverses a graph level by level from a chosen starting node, visiting all neighbours before moving deeper. Figures 30a-33 show BFS initiated from node A, with visited nodes turning green in the order $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$, before the graph is reset and made ready for a new traversal.

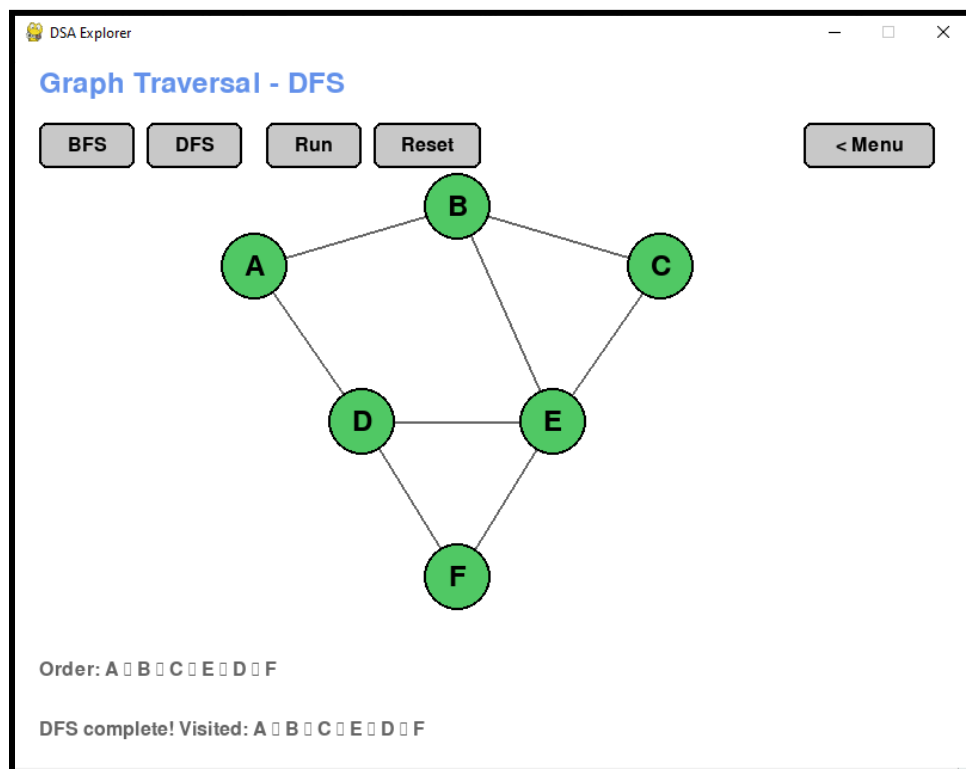
DFS:



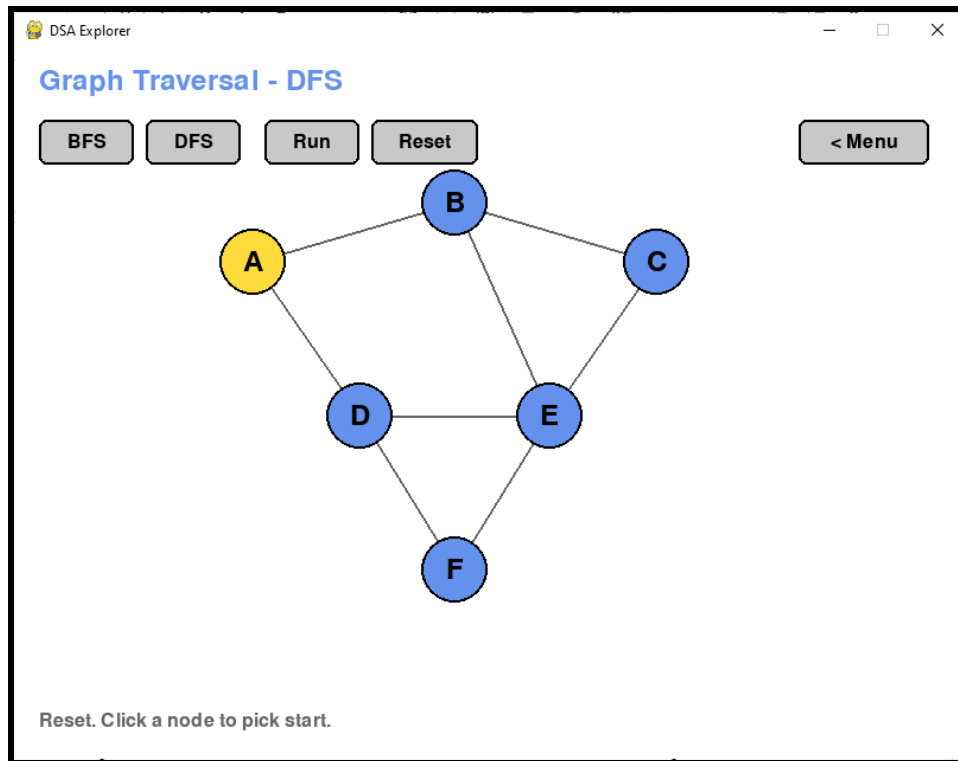
(Figure 30b - DFS selected)



(Figure 34 - DFS running)



(Figure 35 - DFS complete)



(Figure 36 - DFS reset (same as Figure 33))

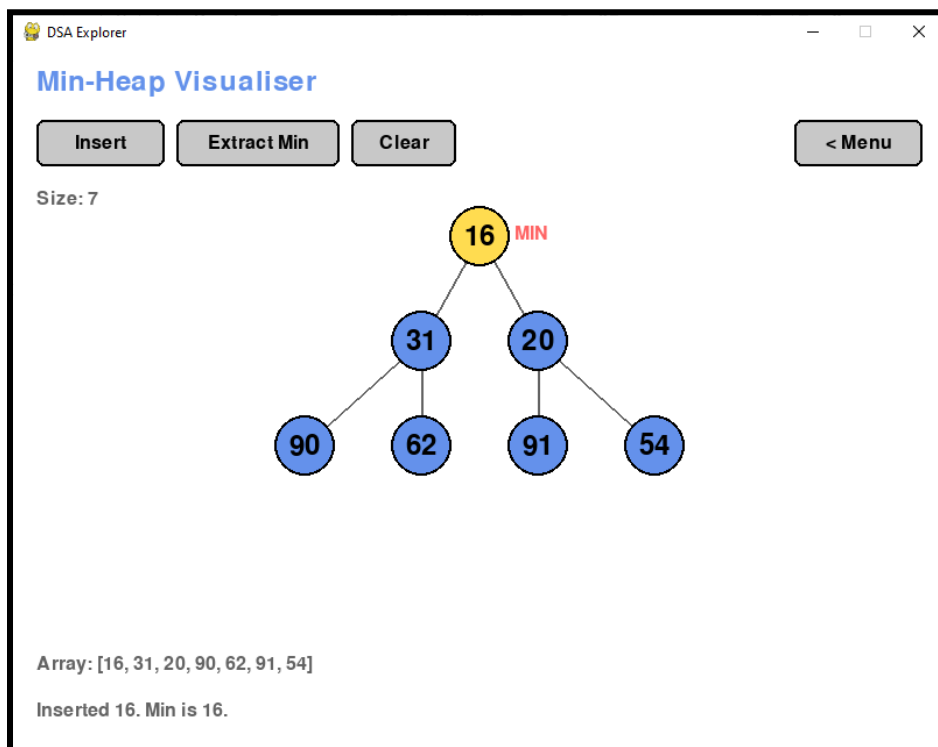
Description:

The DFS module demonstrates Depth-First Search, which traverses as far as possible along each branch before backtracking to explore remaining neighbours. Figures 30b-36 show DFS initiated from node A, with visited nodes turning green in the order $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F$, before the graph resets back to its initial state.

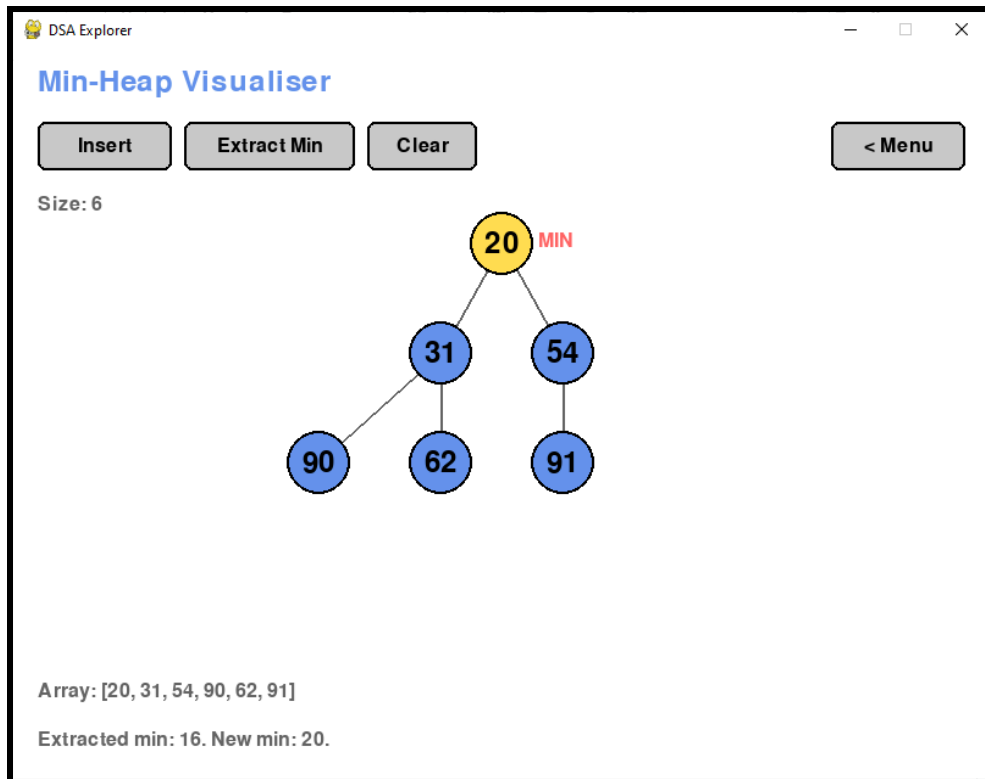
Heap:



(Figure 37 - Heap main menu)



(Figure 38 - Insert)



(Figure 39 - Extract Min)



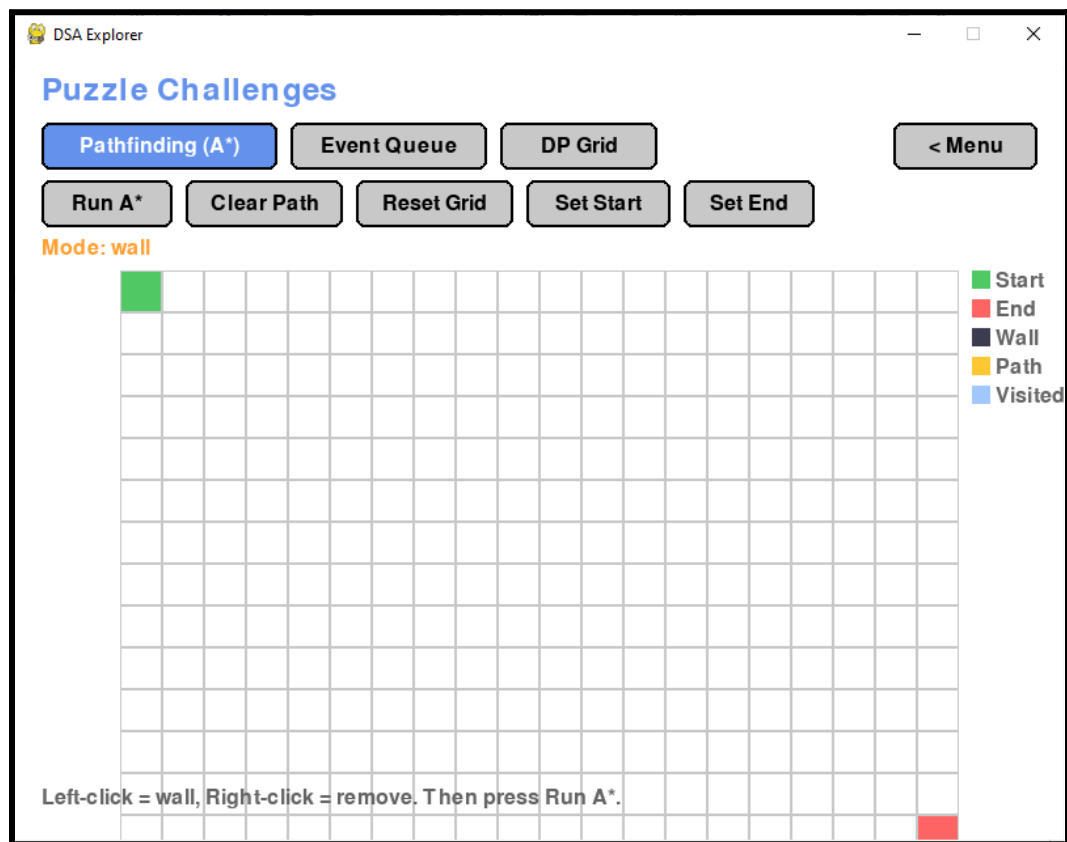
(Figure 40 - Clear)

Description:

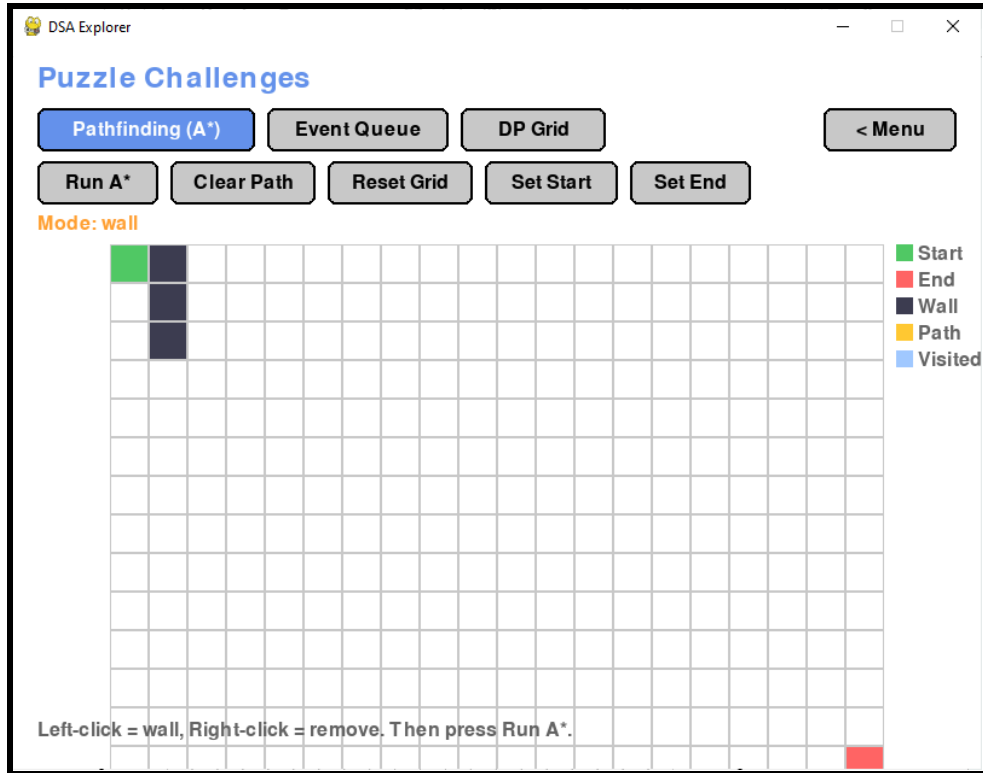
The Min-Heap module visualises a binary tree structure where the smallest value is always maintained at the root, with the heap property restored automatically after every insertion or extraction. Figures 37-40 show seven values being inserted and arranged into a valid min-heap with 16 as the root, the minimum (16) being extracted and the heap restructuring to promote 20 as the new root, and finally the heap being cleared back to empty.

Phase 3:

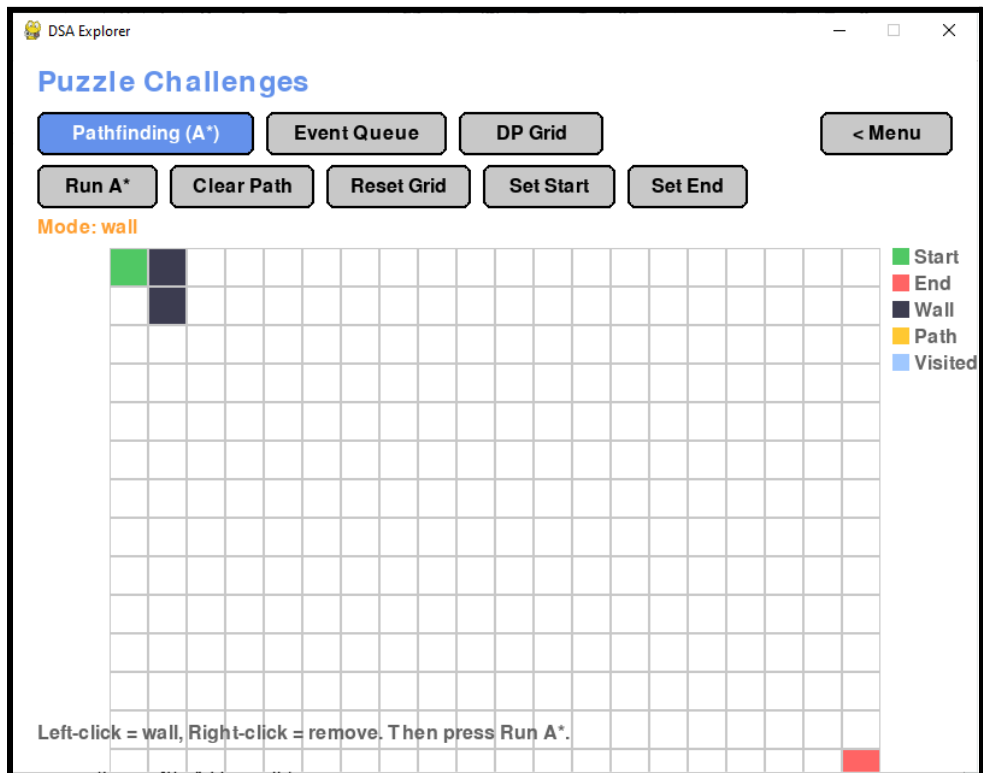
Pathfinding Puzzle:



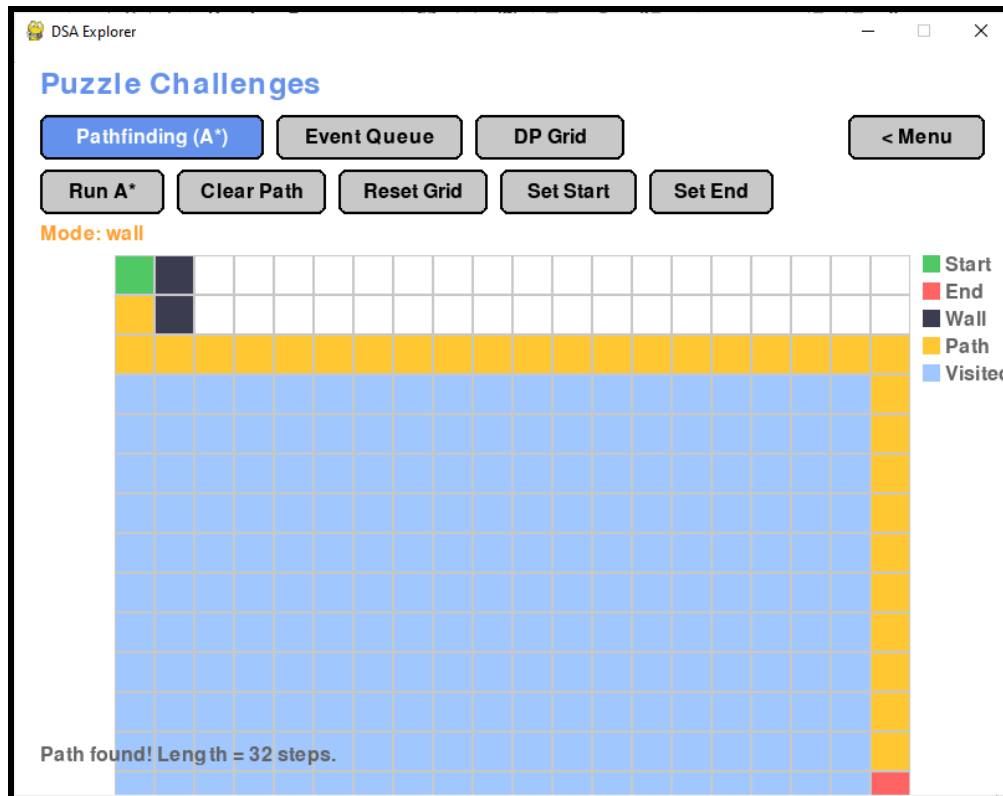
(Figure 41a - Pathfinding (A*) menu)



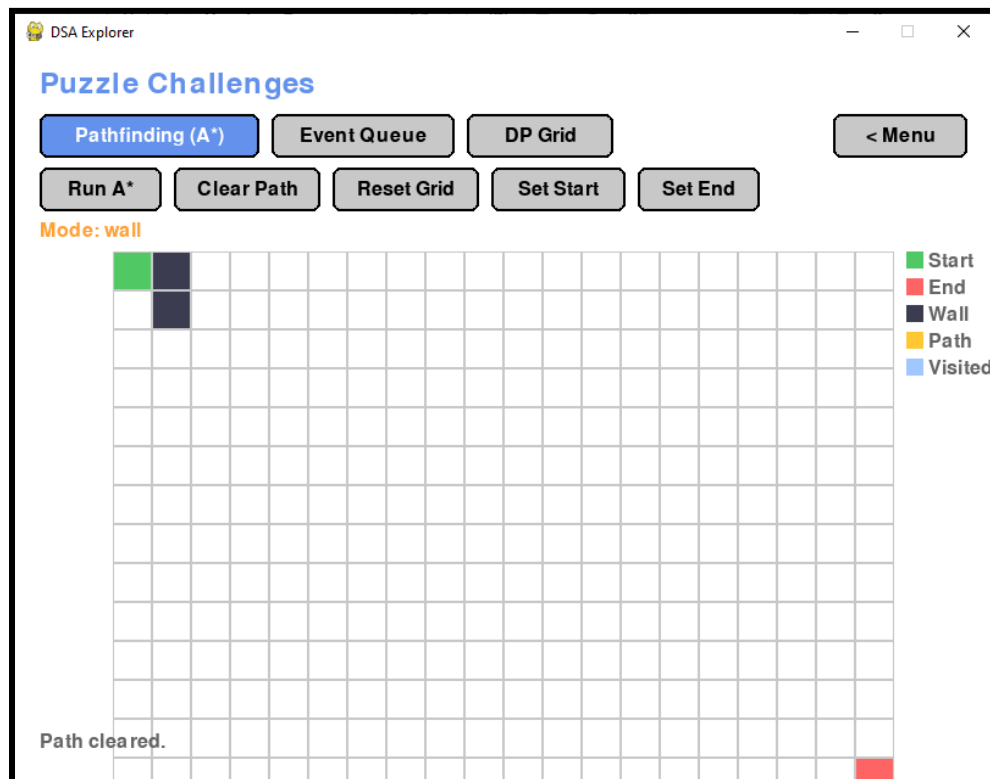
(Figure 42 - Adding path)



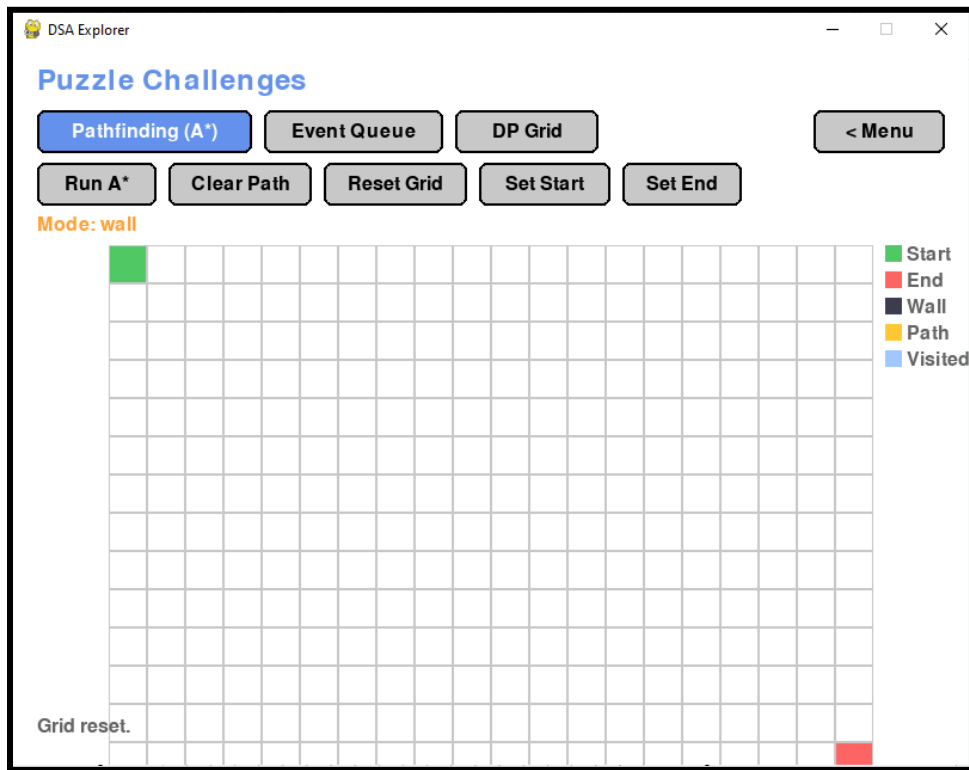
(Figure 43 - Removing path)



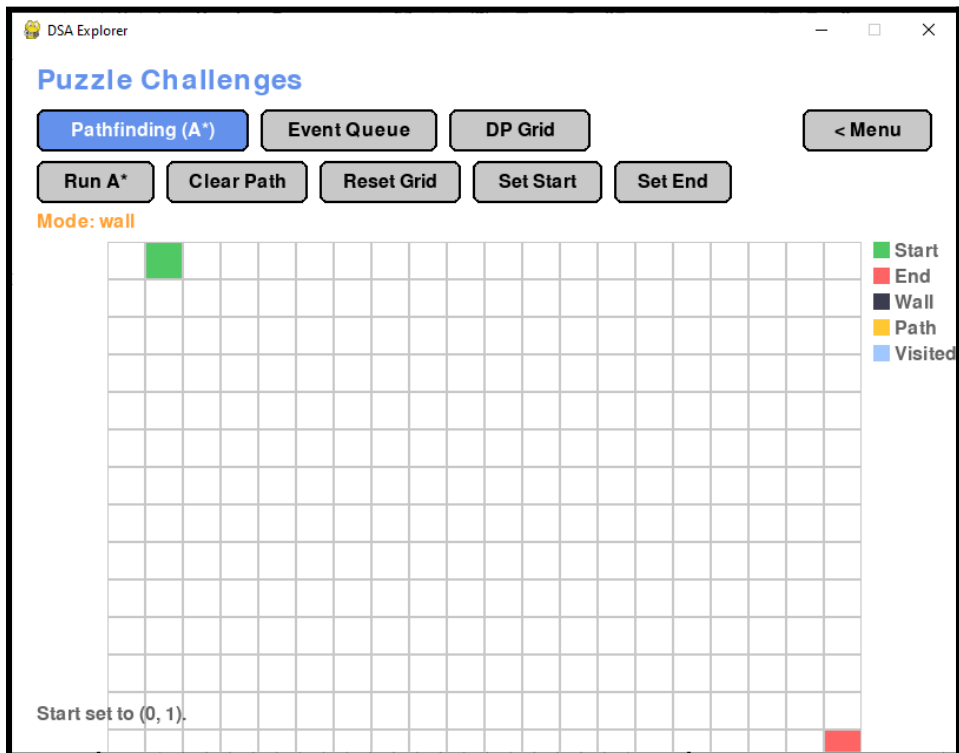
(Figure 44 - Run (A*))



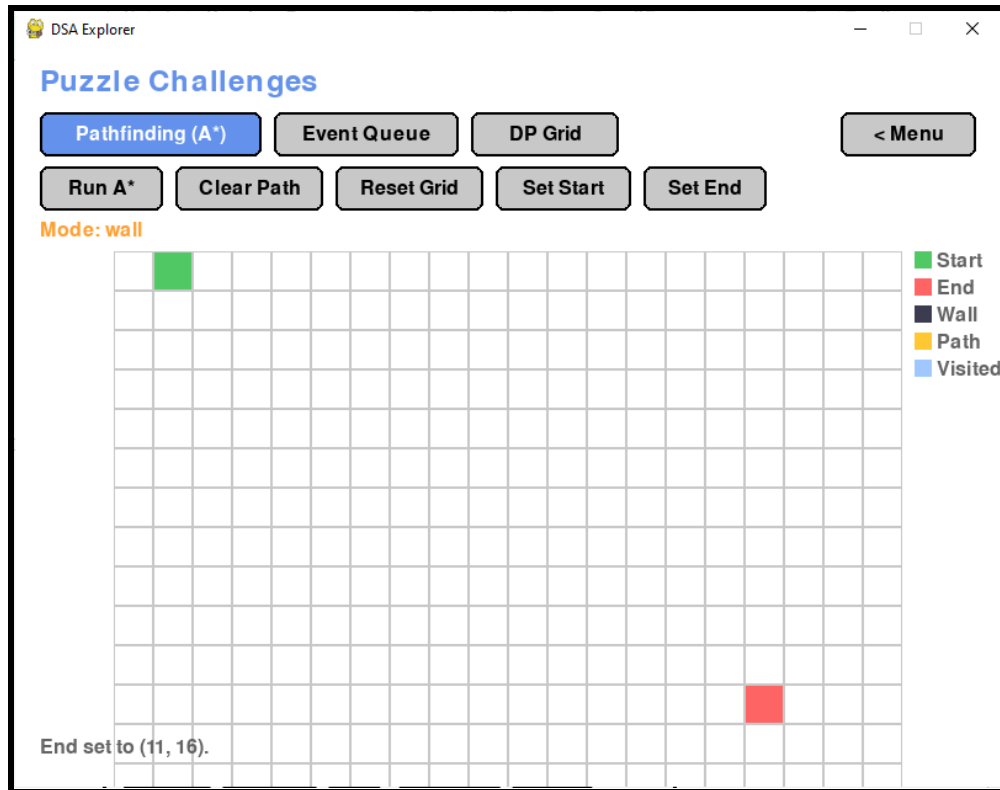
(Figure 45 - Clear Path)



(Figure 46 - Reset Grid)



(Figure 47 - Set Start)

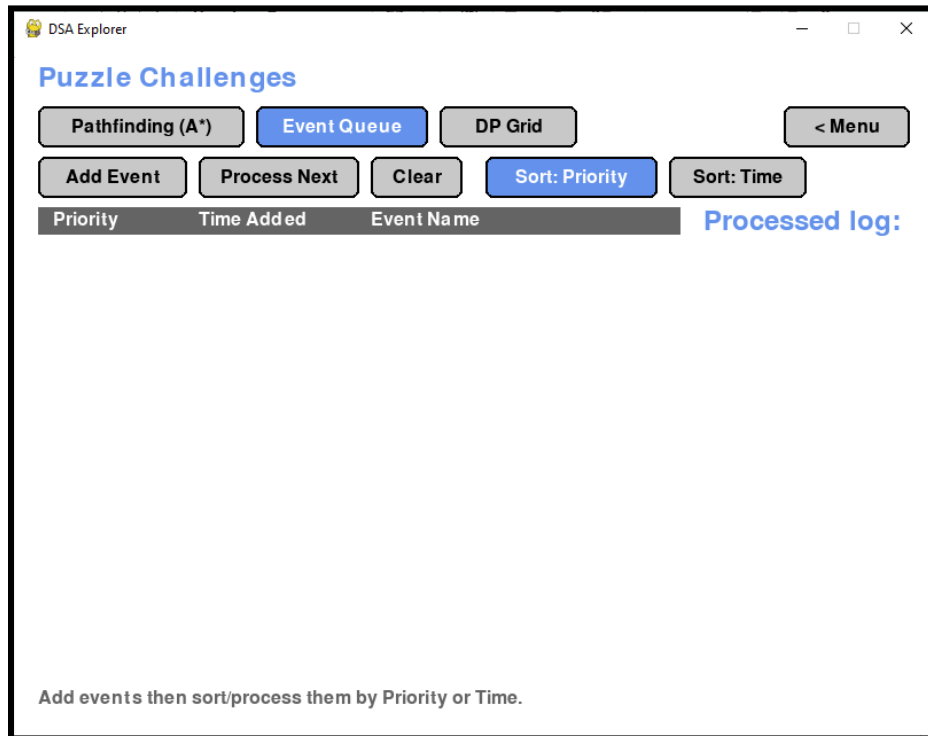


(Figure 48 - Set End)

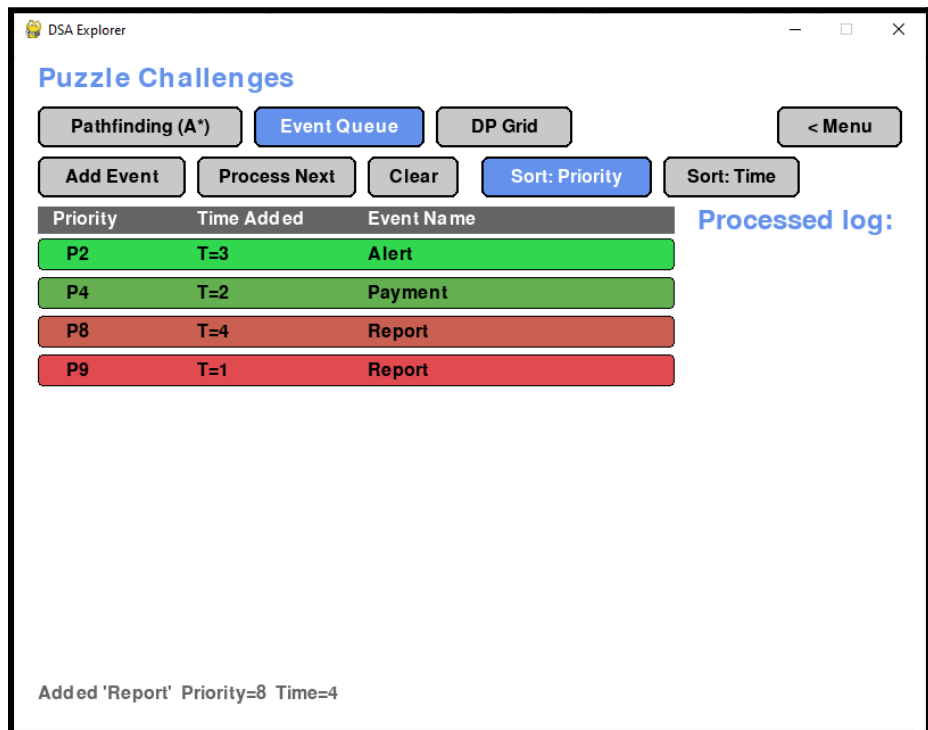
Description:

The Pathfinding puzzle demonstrates the A* algorithm finding the shortest route between a start and end cell on a grid, navigating around user-placed walls. Figures 41a-48 show walls being placed and removed by left and right clicking, A* being run to find a 32-step path highlighted in yellow with visited cells shown in blue, and the start and end positions being repositioned using the Set Start and Set End buttons.

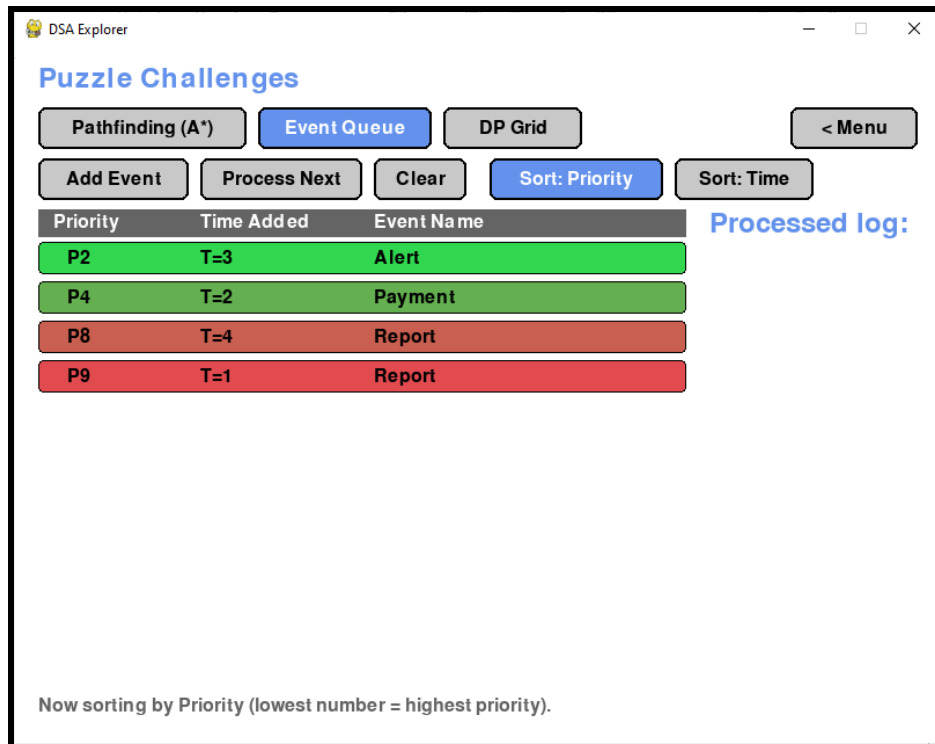
Event Queue Simulator:



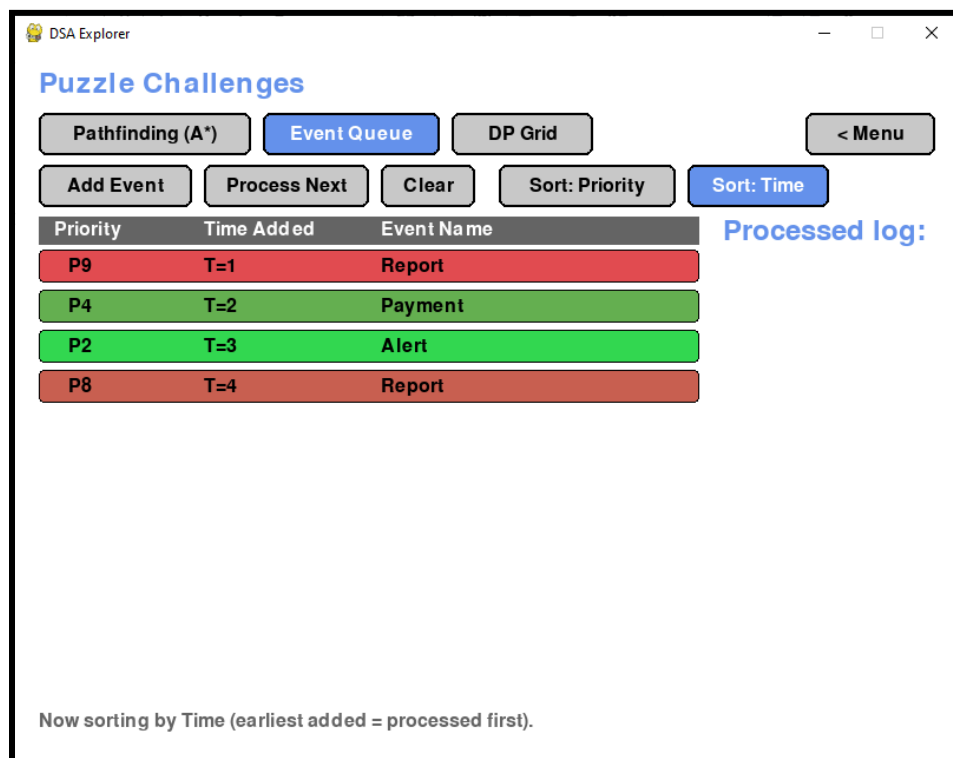
(Figure 41b - Event Queue menu)



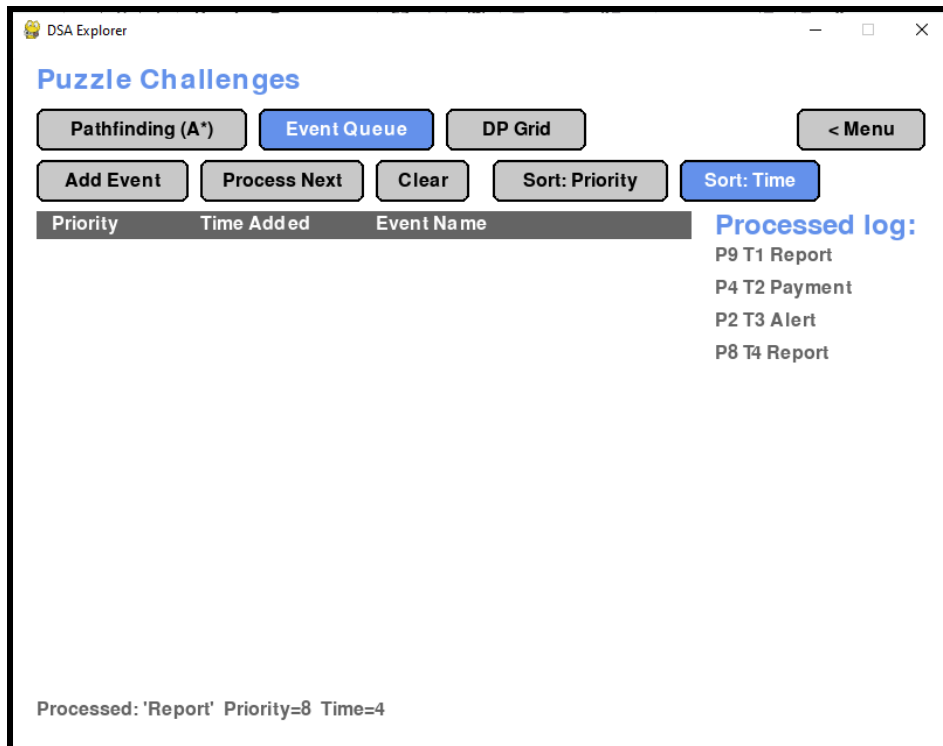
(Figure 49 - Add Event)



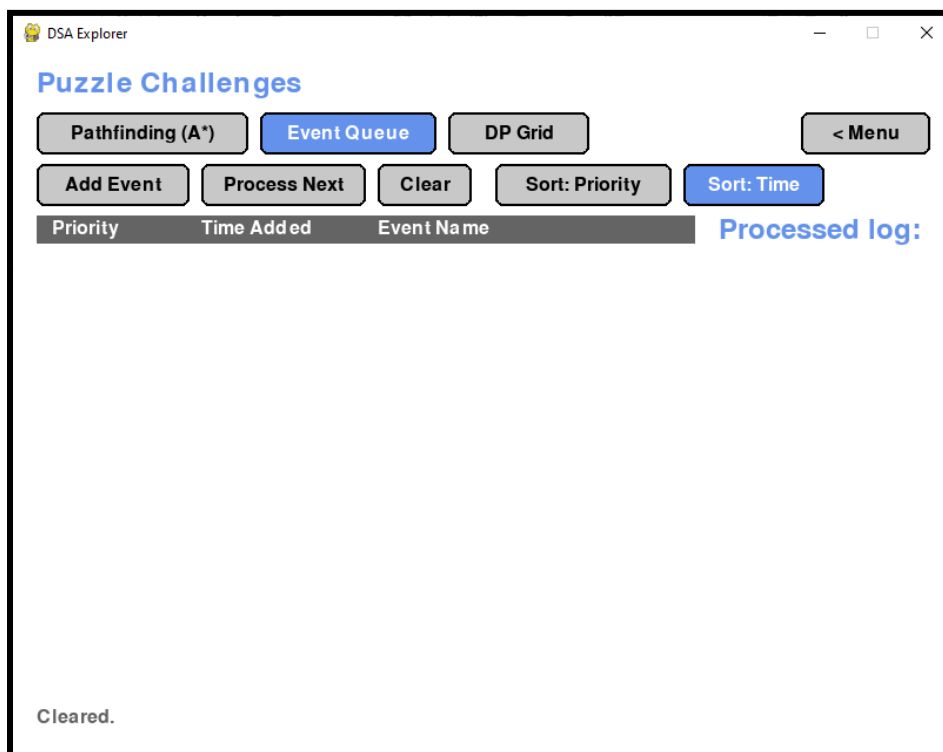
(Figure 50 - Sort: Priority)



(Figure 51 - Sort: Time)



(Figure 52 - Process Next)

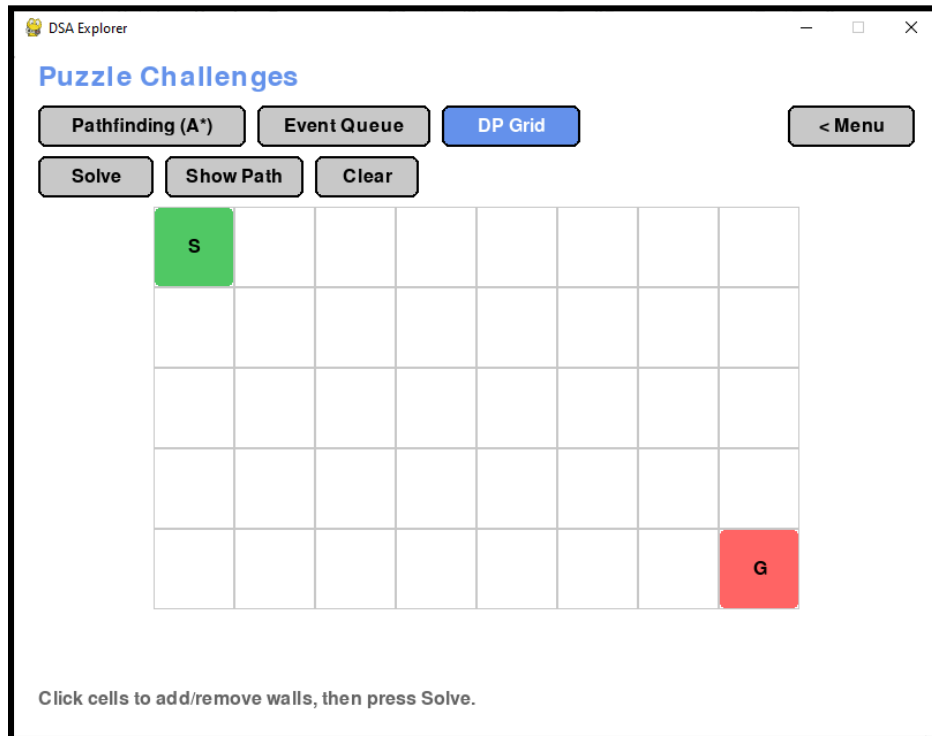


(Figure 53 - Clear)

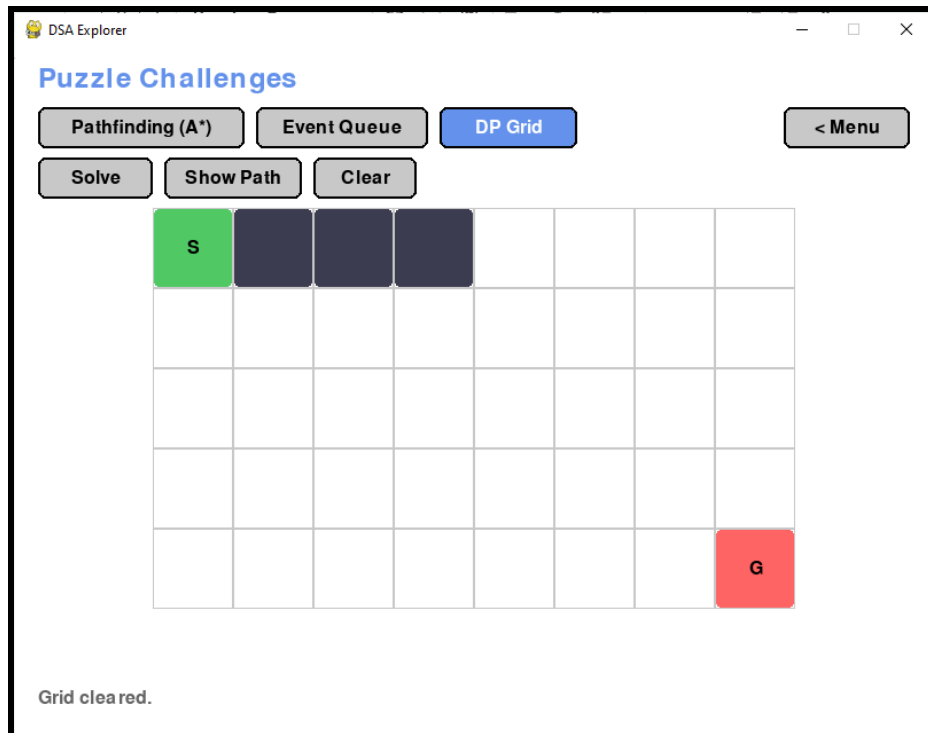
Description:

The Event Queue simulator demonstrates a priority queue where events can be sorted and processed either by priority level or by their time. Figures 41b-53 show four events being added with different priorities and times, the queue being resorted by priority (lowest number first) and then by time (quickest/earliest first), events being processed one at a time and logged to the processed log panel, and the queue being fully cleared.

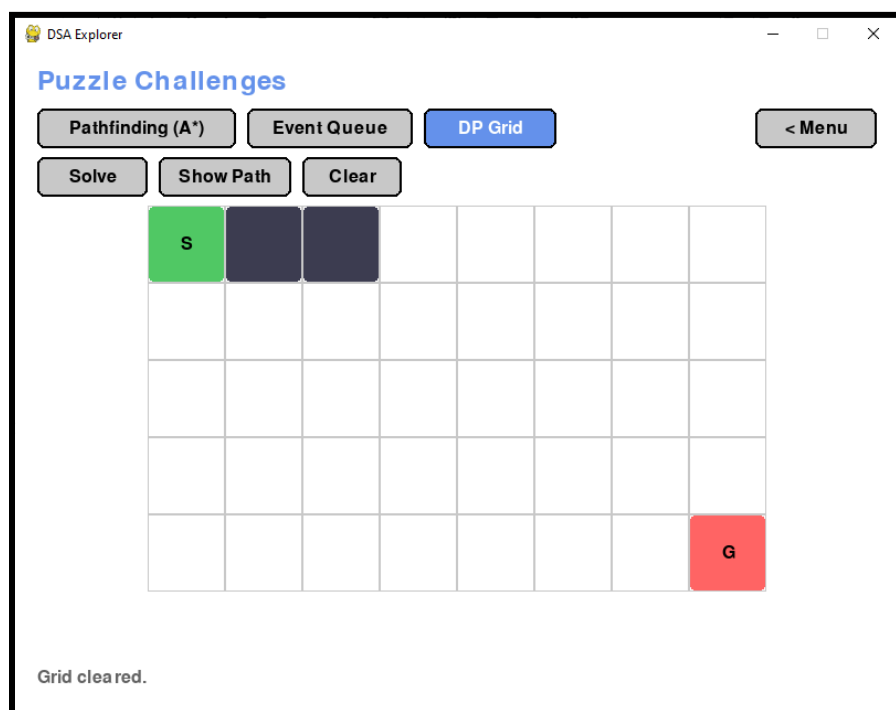
Dynamic Programming Puzzle:



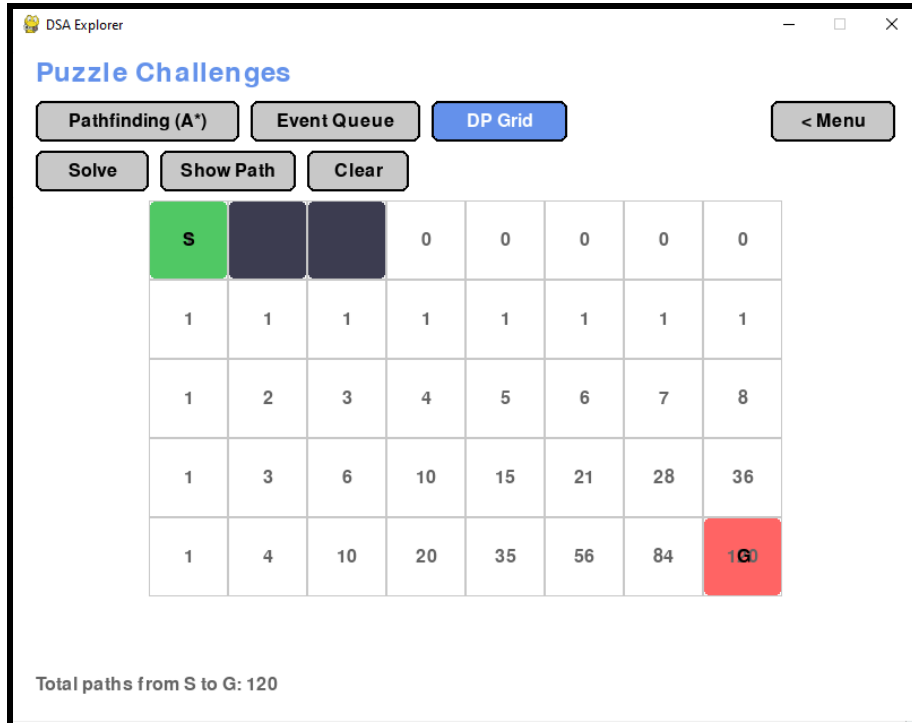
(Figure 41c - DP Grid menu)



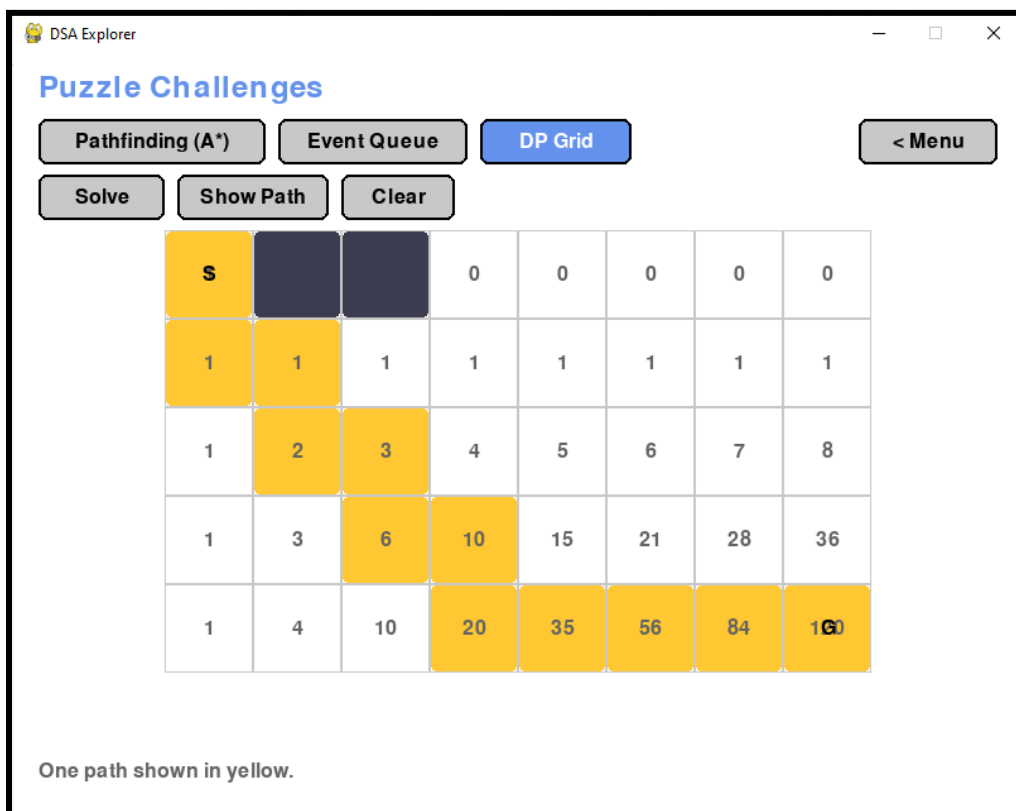
(Figure 54 - Add obstacle)



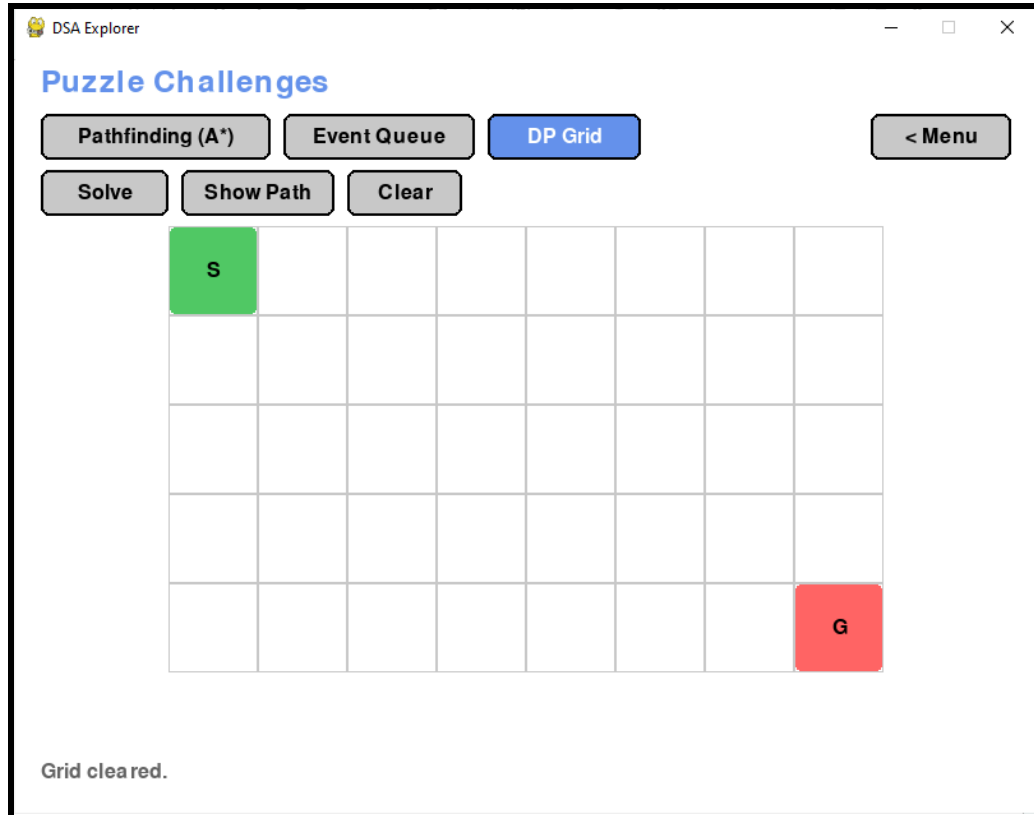
(Figure 55 - Remove obstacle)



(Figure 56 - Solve)



(Figure 57 - Show Path)



(Figure 58 - Clear)

Description:

The Dynamic Programming puzzle demonstrates a grid-based pathfinding problem where the application calculates the total number of unique paths from the start (S) to the goal (G), filling each cell with a running count using bottom-up dynamic programming. Figures 41c-58 show obstacles being placed and removed by clicking cells, the Solve function populating the grid with path counts (120 in this example), Show Path highlighting one valid route in yellow, and Clear resetting the grid back to its empty state.

Test Cases:

Module 1 - Data Structures Test Case:

Test Case	Description	Expected Result
Stack push/pop sequence	Push 3 items, pop 2	Final stack size = 1, correct order

Queue enqueue/dequeue	Enqueue 4 items, dequeue 3	FIFO order maintained
Linked list insertion at pos	Insert node with value 10 at position 2	Node present in correct location in list
BST insert and inorder traversal	Insert [50, 30, 70] inorder traversal	Traversal order: 30, 50, 70
Stack empty pop	Attempt to pop from an empty stack	Message displays "Stack is empty - nothing to pop!", stack remains 0
Linked list reverse	Insert Head 3 times (1, 2, 3), then click reverse	List order changes from [3 → 2 → 1] to [1 → 2 → 3], HEAD label moves to node 1
BST duplicate insert	Insert value 50 twice into the BST	Only one node with the value 50 appears (duplicates are silently ignored per the insert logic)

Module 2 - Sorting Algorithm Test Case:

Test Case	Description	Expected Result
Bubble sort correctness	Sort array of [5, 3, 8, 1, 2]	Sorted array: [1, 2, 3, 5, 8]
Selection sort animation	Ensures colours highlight compared elements	Highlight changes each step correctly
Already sorted array	Run bubble sort on an already sorted array	Array remains [1, 2,..., n], no swaps occur, all bars turn green on final step
Selection sort correctness	Run selection sort on [5, 3, 8, 1, 2]	Sorted output: [1, 2, 3, 5, 8]
Shuffle generates new array	Click Shuffle, then Start	A new random array is loaded and sorted correctly from the new starting state

Module 3 - Graph Traversal Test Case:

Test Case	Description	Expected Result
BFS traversal from 'A'	Visit nodes reachable from 'A'	Nodes visited in correct BFS order
DFS traversal from 'C'	DFS order matches expected path	Traversal path matches theoretical DFS
Interactive start node	Clicking different nodes	BFS restarts correctly from chosen node
BFS visits all nodes	Run BFS from 'A' on the full graph	All 6 nodes (A, B, C, D, E, F) are visited , none skipped
DFS from order 'A' order	Run DFS starting from node 'A'	Traversal order $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F$ (based on reverse-sorted neighbour push)
Switch traversal mode mid-graph	Run BFS from 'A' until complete, switch to DFS, click 'A' again	Graph resets, DFS runs fresh from A with correct DFS ordering

Module 4 - Heap Test Case:

Test Case	Description	Expected Result
Min-heap root property	Insert multiple values (e.g. 40, 10, 70, 5) via Insert button	The root note (highlighted yellow) always displays the smallest value inserted, "MIN" label sits beside it
Extract min removes root	Insert 3 values, then click Extract Min button	The smallest value is removed, the heap restructures via sift-down, and the message reads "Extracted min: X. New min: Y."
Extract from empty heap	Click Extract Min button on an empty heap	Message displays "Heap is empty!" and the heap remains empty with no crash

Module 5 - Puzzles Test Case:

Test Case	Description	Expected Result
A* finds path	On the pathfinding tab, leave grid clear and click Run A*	A path is highlighted in yellow, from the green Start cell (0,0) to the red End cell (13, 19), and the message displays "Path found! Length = X steps."
Event queue priority sort	On the Event Queue tab, add 3+ events and ensure Sort: Priority is active, then click Process Next	The event with the lowest priority number is processed first and logged in the format "PX TY EventName"
DP Grid no path	On the DP Grid tab, block an entire row of cells to wall off Start from Goal, then click Solve	The message displays "Total paths from S to G: 0" and no yellow path appears when Show Path is clicked